

Computer Vision

Diffusion, Perception, and Safety

Nikita Durasov, 13.02.2024

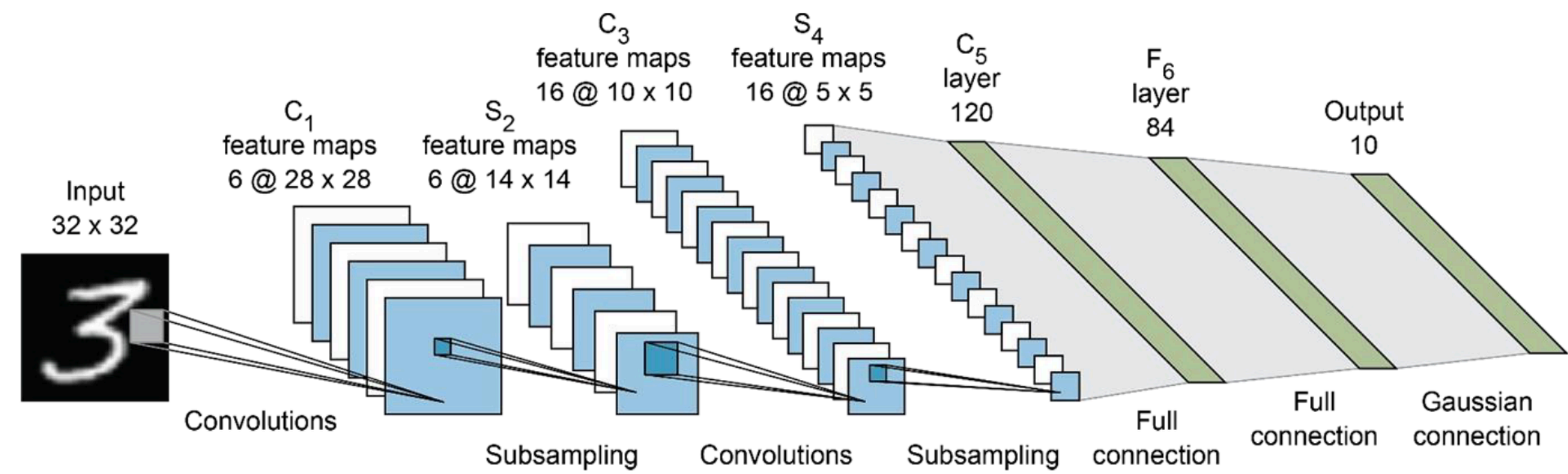
Outline

Main Topics

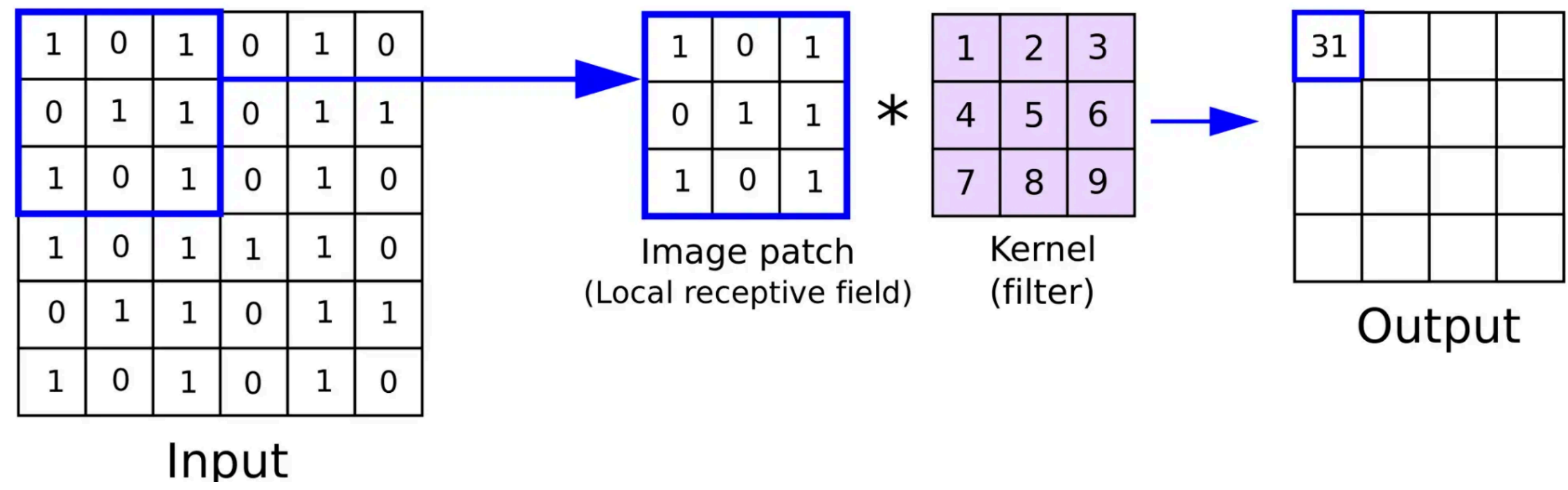
- **Diffusion Models**
- **Visual Perception**
- **Generalization**
- **Uncertainty Estimation**

General Terms

Convolutions



iq.opengenus.org



General Terms

Convolutions

1 _{x1}	1 _{x0}	1 _{x1}	0	0
0 _{x0}	1 _{x1}	1 _{x0}	1	0
0 _{x1}	0 _{x0}	1 _{x1}	1	1
0	0	1	1	0
0	1	1	0	0

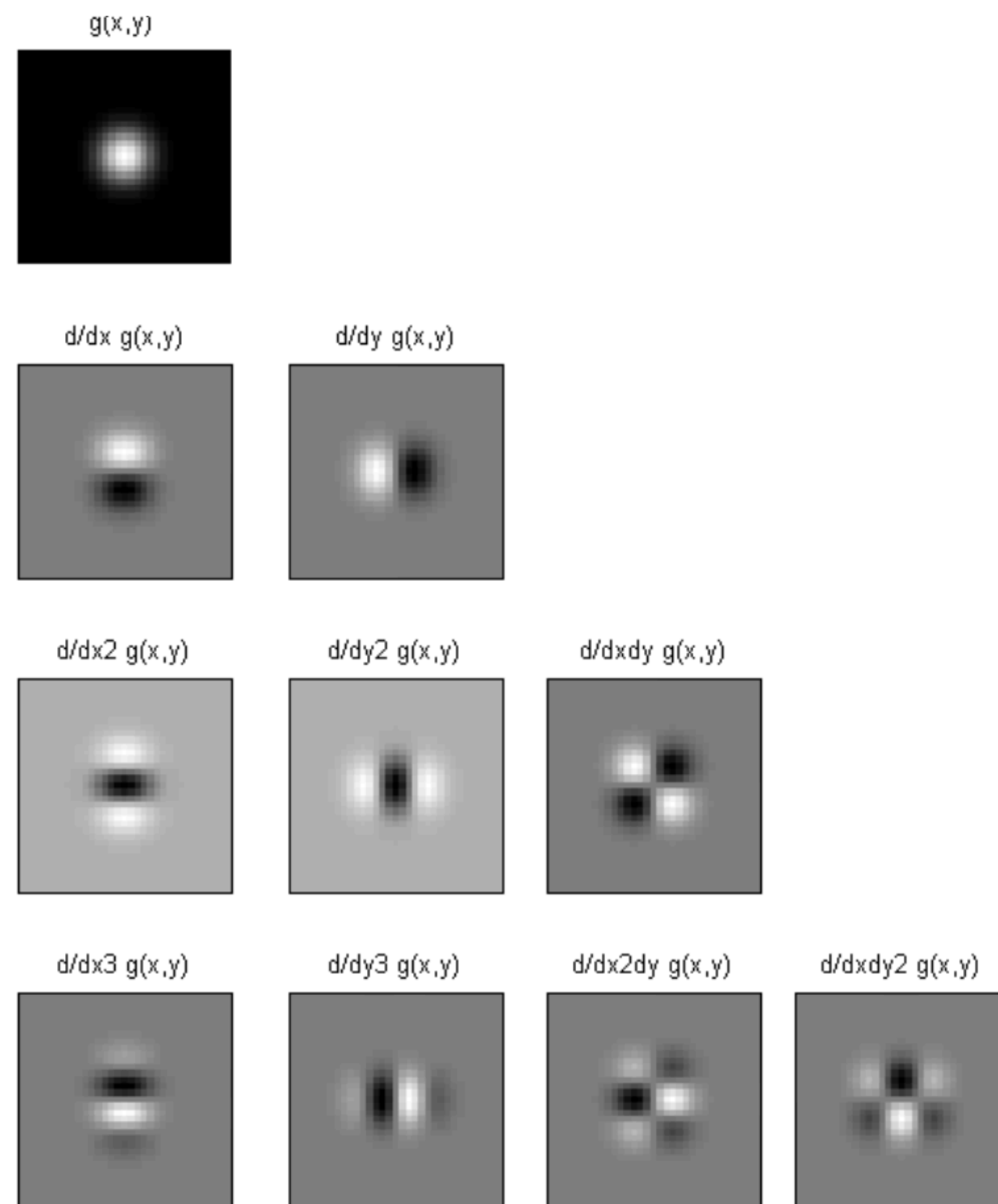
Image

4		

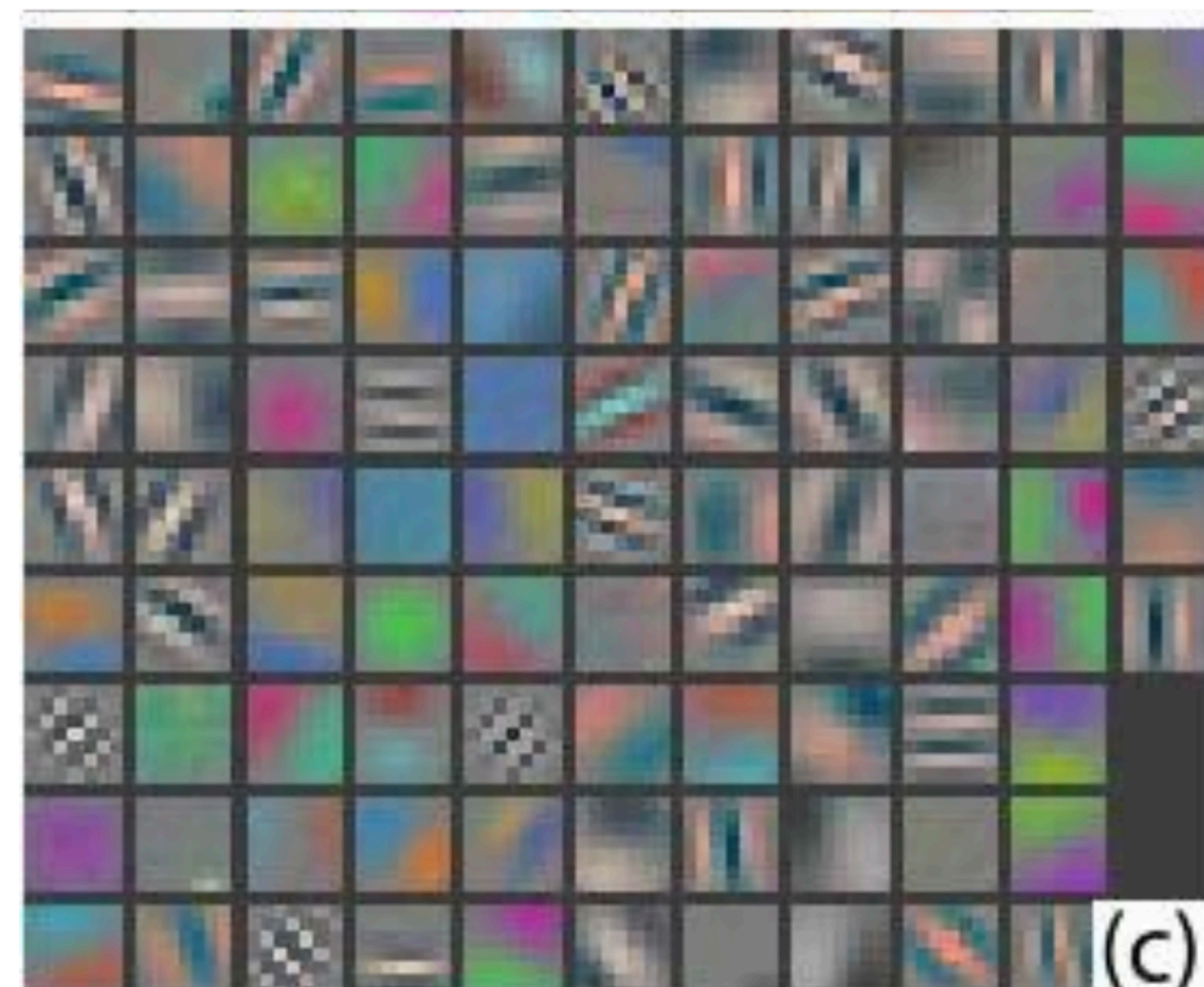
Convolved
Feature

General Terms

Convolutions



Derivatives



Learned filters

General Terms

Convolutions

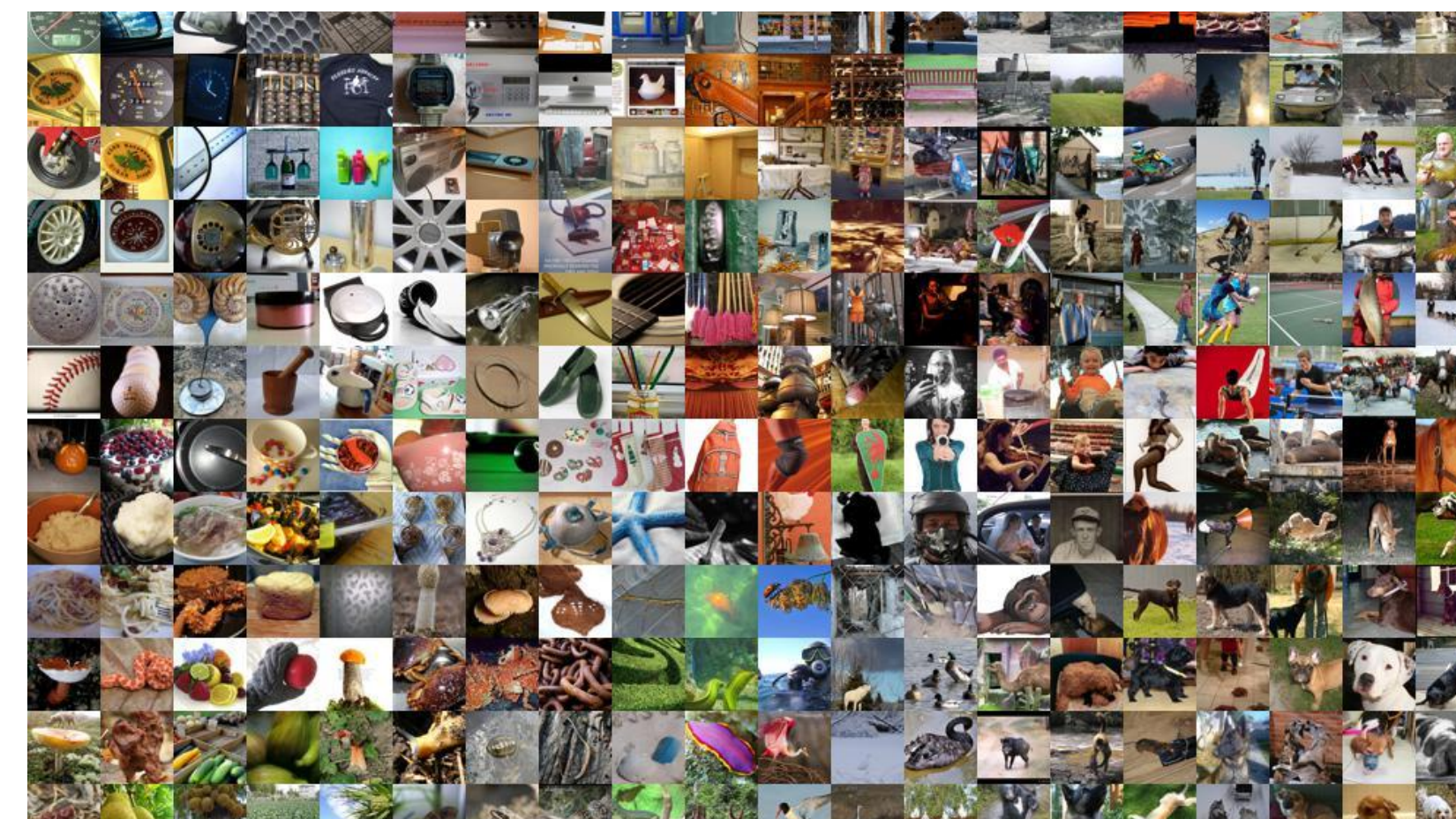
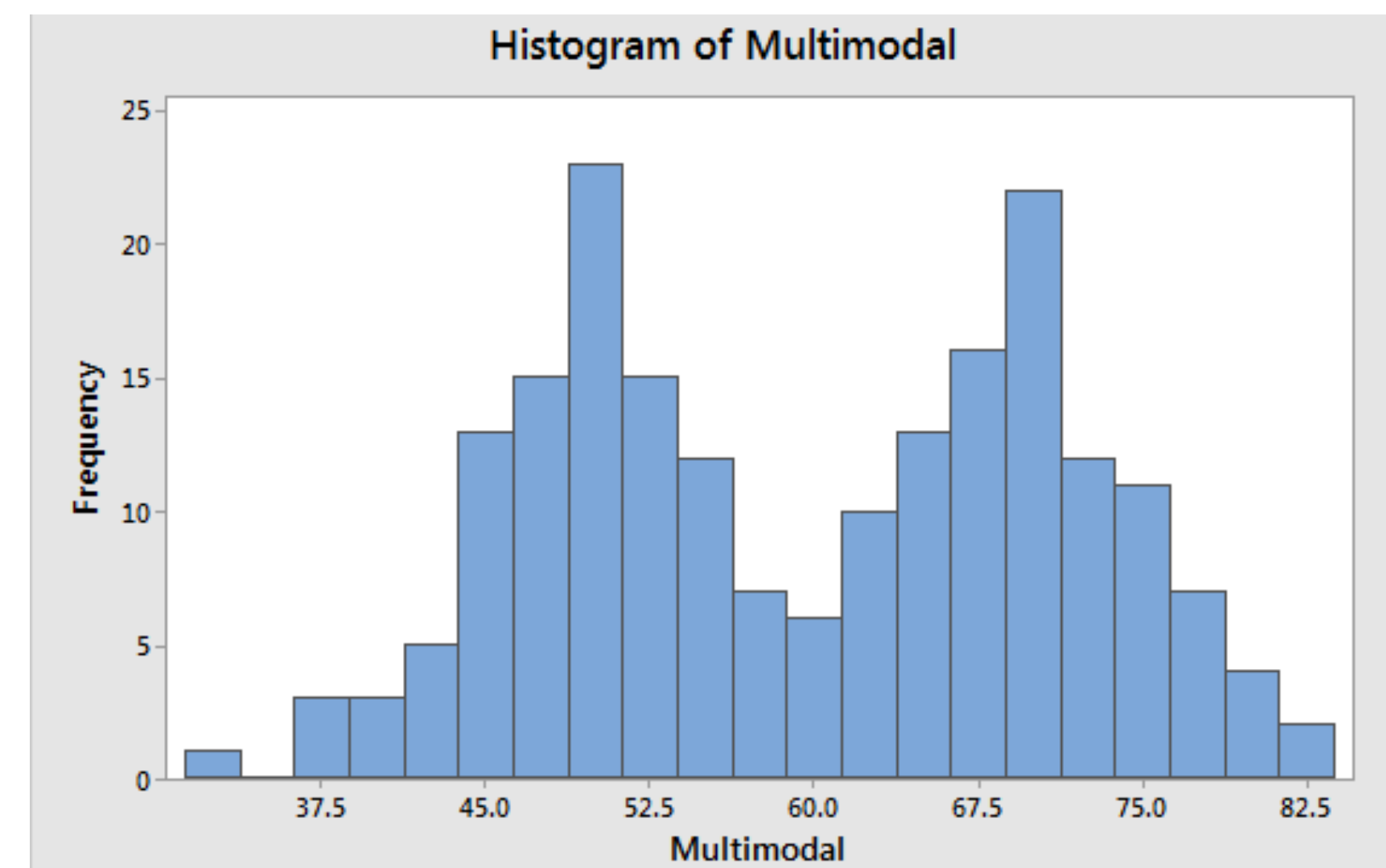
Since the introduction of convolutions (~1980),
many fascinating advancements have emerged—let's dive into them!

Part 1: Diffusion Models

Diffusion Models

Probabilistic/Generative Modelling

- Suppose we have a training data distribution $p(x)$
- The objective of generative modeling is to learn this distribution and generate samples from it.

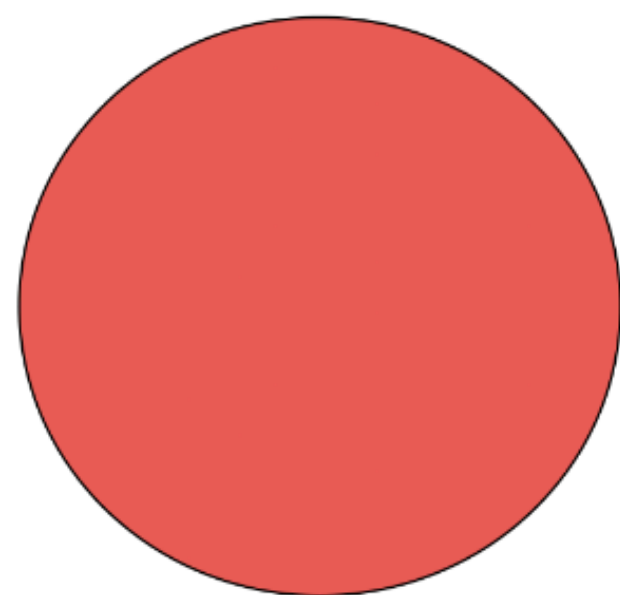


Diffusion Models

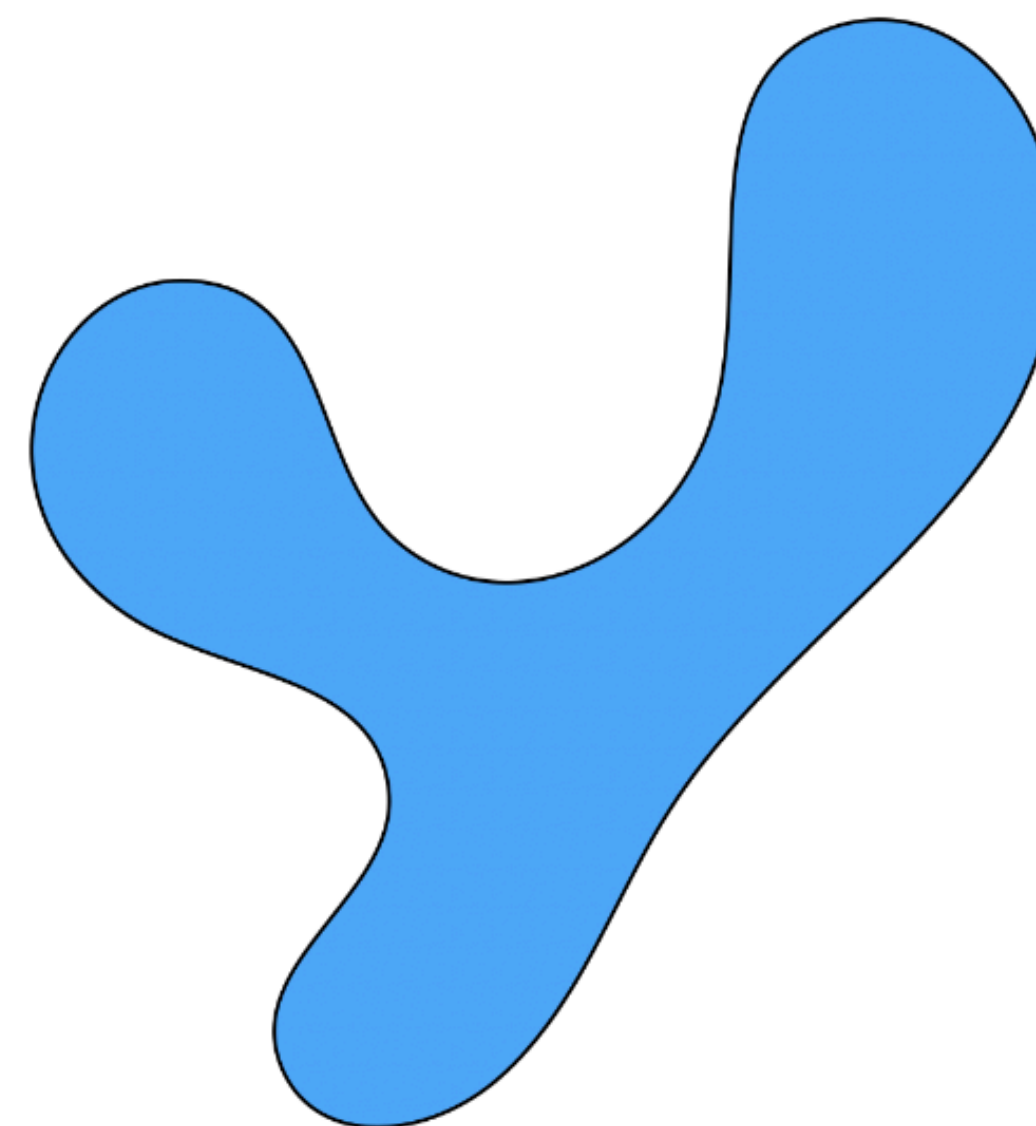
Sampling from Noise

Source distribution

Target distribution



$p(z)$

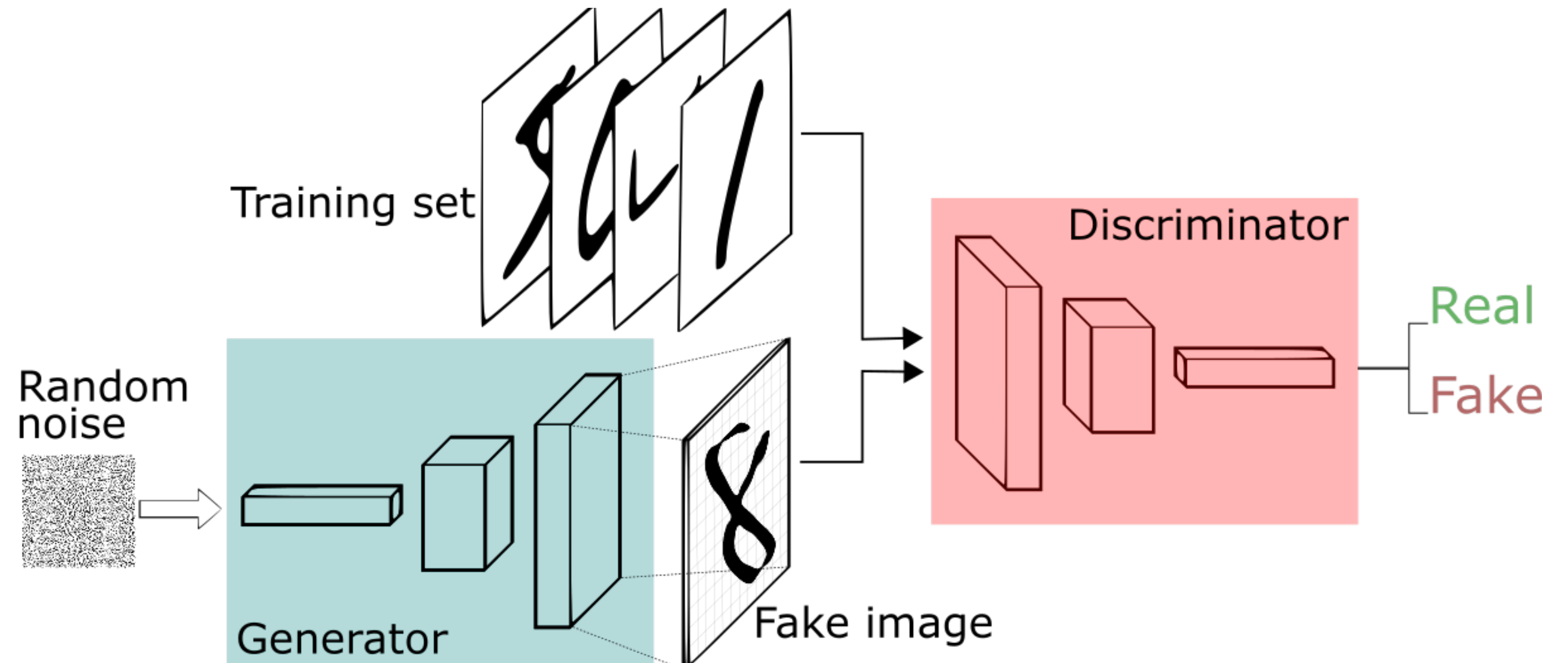


$p(x)$

Generative Models

Generative Adversarial Networks (GANs)

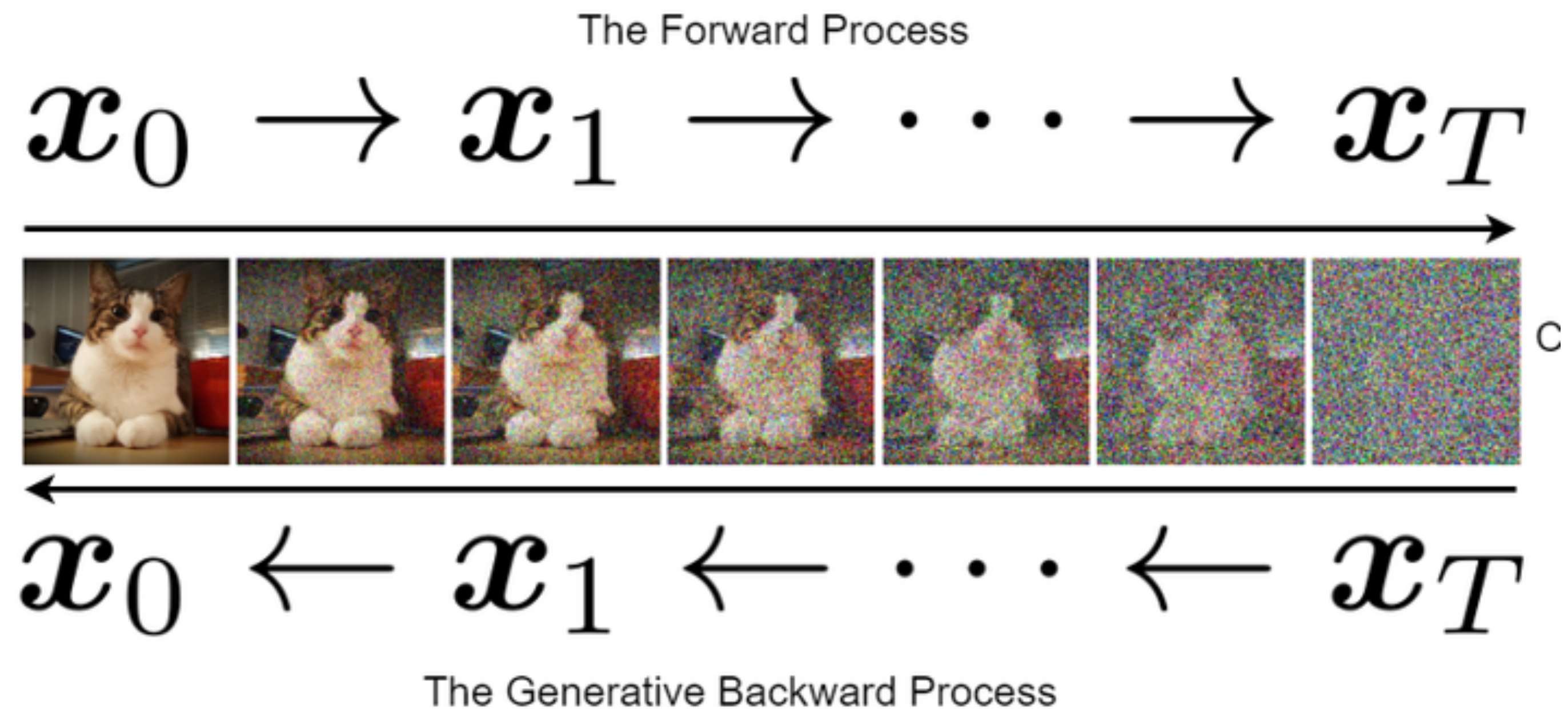
- Map a latent noise vector directly to the target distribution in a single step
- Has lots of issues with training (instability)



Diffusion Models

General Idea

- **Idea:** Estimating and analyzing small step sizes is more feasible and manageable than making a single leap from random noise to the learned distribution.
- Iteratively transform a well-known and simple base distribution (such as a Gaussian) into the target data distribution



Diffusion Models

Forward Process

$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t}\mathbf{x}_{t-1}, \beta_t\mathbf{I}) \quad q(\mathbf{x}_{1:T}|\mathbf{x}_0) = \prod_{t=1}^T q(\mathbf{x}_t|\mathbf{x}_{t-1}) \quad \{\beta_t \in (0, 1)\}_{t=1}^T$$

The forward process of diffusion represents adding random noise to the input from Gaussian distribution with changing variance

Diffusion Models

Implementing Forward Process

- Sample an image from the dataset
- Sample noise (from a standard normal distribution) $\epsilon \sim \mathcal{N}(0, I)$
- Scale the image by: $\sqrt{\alpha_t} : \sqrt{\alpha_t} x_0$
- Add: $\sqrt{1 - \bar{\alpha}_t} \epsilon : \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$

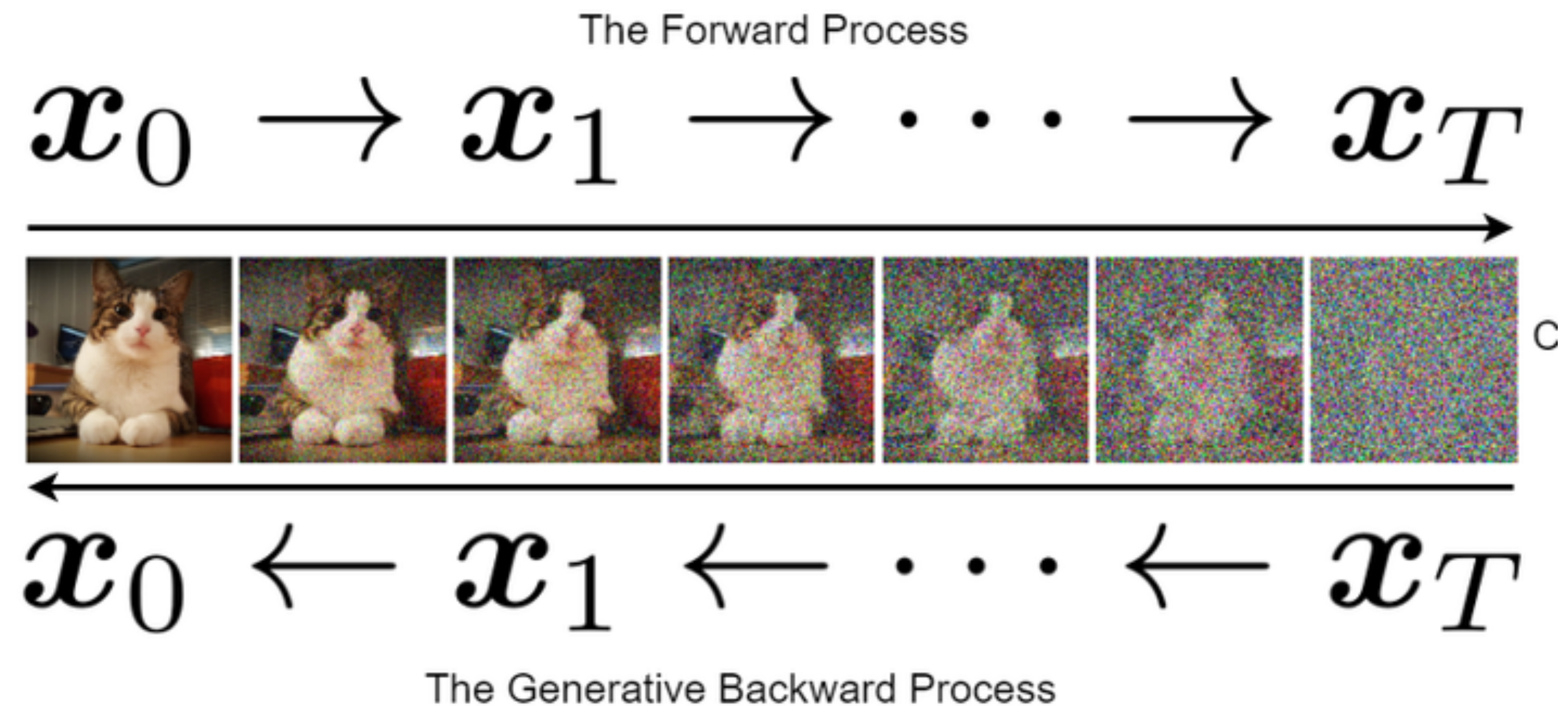


Diffusion Models

Implementing Backward Process

- The objective of a diffusion model is to learn the reverse denoising process, gradually reconstructing the original data by inverting the forward process step by step
- This makes the reverse process resemble the generation of new data starting from pure random noise!

Forward distribution is known

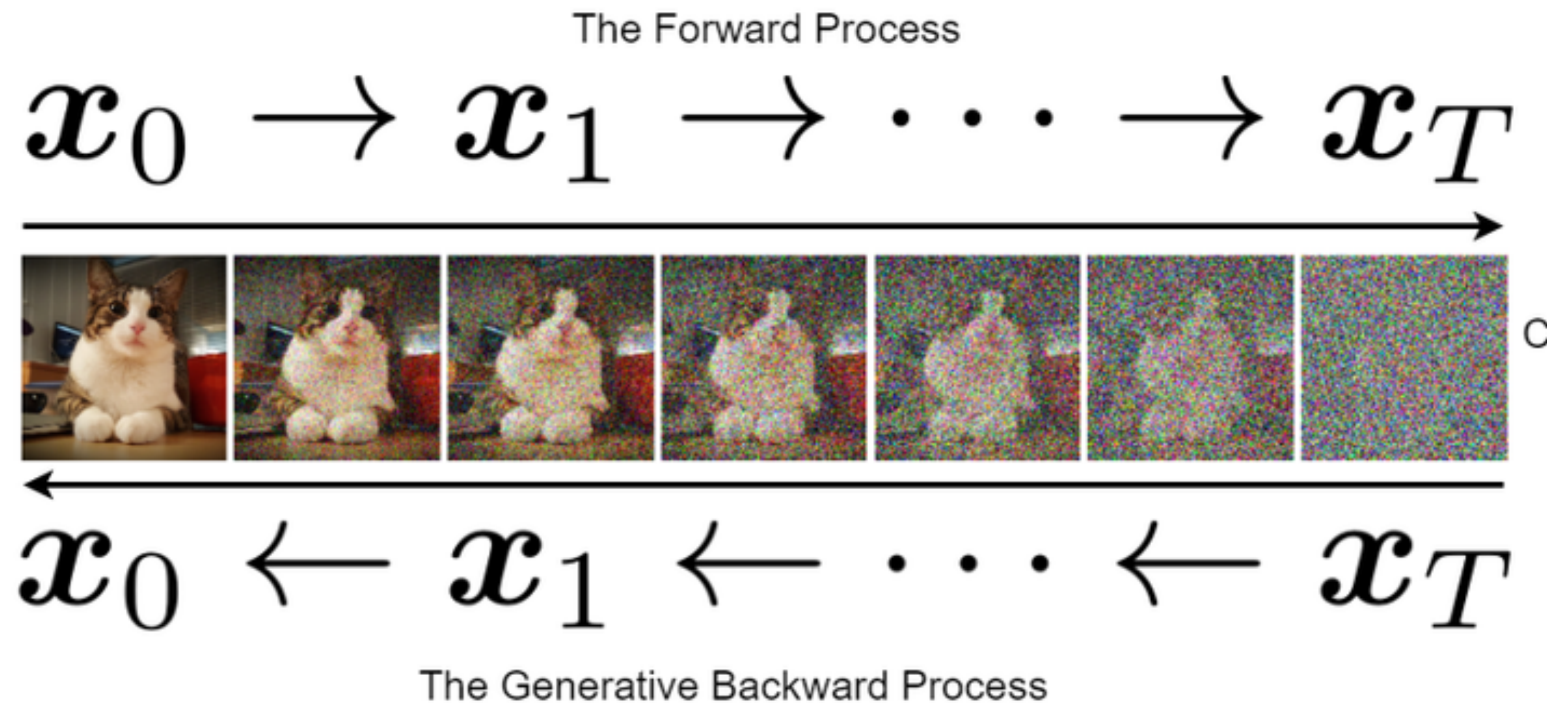


Backward distribution has to be learned!

Diffusion Models

Backward Distribution

- It turns out that if the forward steps are small enough, each step of the reverse process can also be approximated by a Gaussian distribution.
- Therefore, we can parameterize the learned reverse process as a Gaussian distribution with a mean that is modeled by a neural network.



Diffusion Models

Learning Backward Pass

- Our goal is to train the network that parameterizes the mean of the data distribution:

$$p_{\theta}(\mathbf{x}_{t-1} | \mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \boldsymbol{\mu}_{\theta}(\mathbf{x}_t, t), \mathbf{I})$$

Less noise



More noise



Diffusion Models

Training

- **Step #1:** Sample an image from the training dataset and add noise to it
- **Step #2:** Given noisy image, add a bit more noise



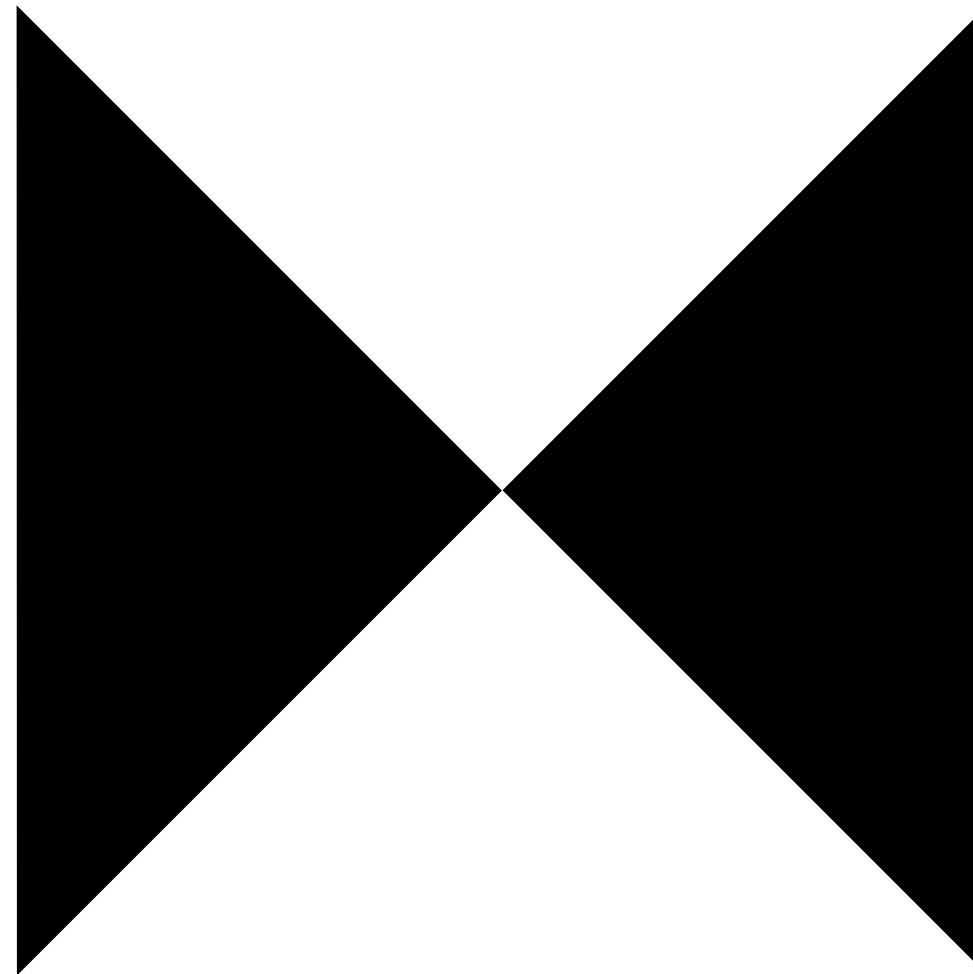
Diffusion Models

Training

Input



UNet



Output



Diffusion Models

Training Algorithm

Algorithm 1 Training

- 1: **repeat**
 - 2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
 - 3: $t \sim \text{Uniform}(\{1, \dots, T\})$
 - 4: $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
 - 5: Take gradient descent step on
$$\nabla_{\theta} \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_{\theta}(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}, t) \right\|^2$$
 - 6: **until** converged
-

Diffusion Sampling

Sampling Algorithm

Algorithm 2 Sampling

```
1:  $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$   
2: for  $t = T, \dots, 1$  do  
3:    $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$  if  $t > 1$ , else  $\mathbf{z} = \mathbf{0}$   
4:    $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left( \mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$   
5: end for  
6: return  $\mathbf{x}_0$ 
```

Diffusion Models

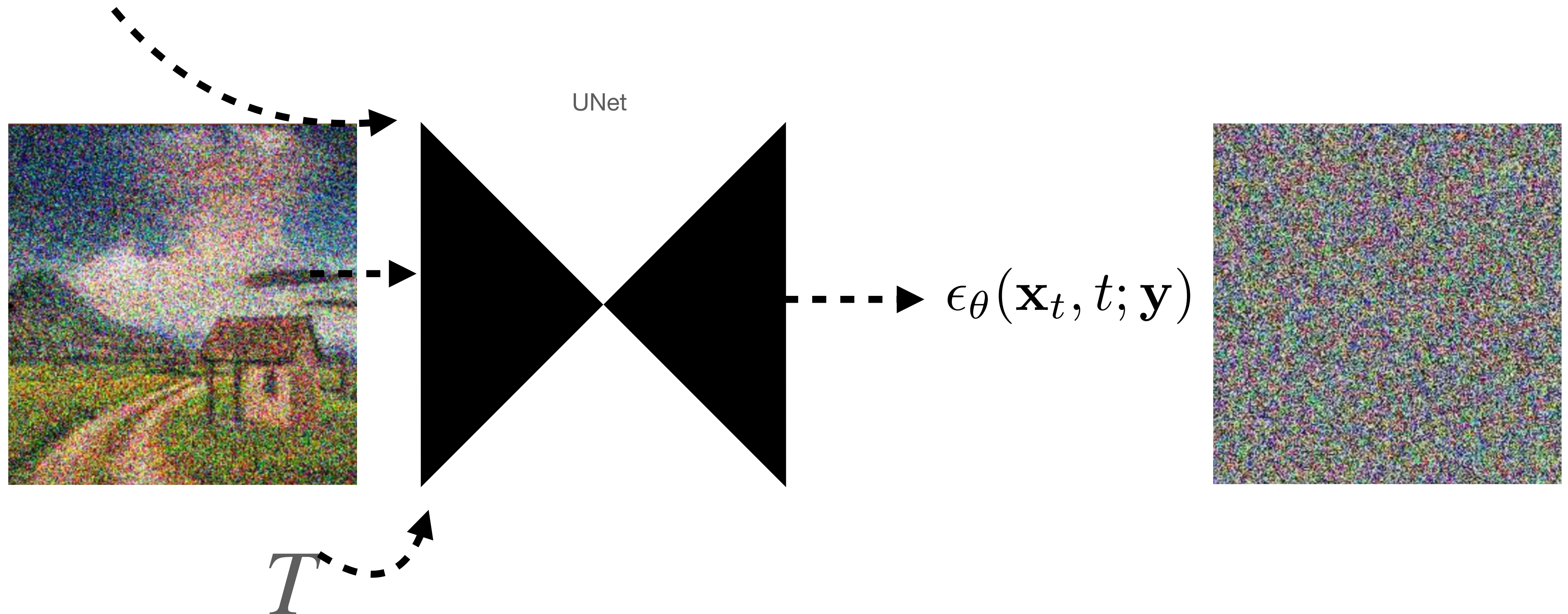
Explicit Conditioning

“A rustic chapel by a winding road, set in a serene countryside under a dramatic sky.”

Diffusion Models

Explicit Conditioning

“A rustic chapel by a winding road, set in a serene countryside under a dramatic sky.”



Diffusion Models

Explicit Conditioning

blue cat

IP



www.shutterstock.com - 150153689



Diffusion Models

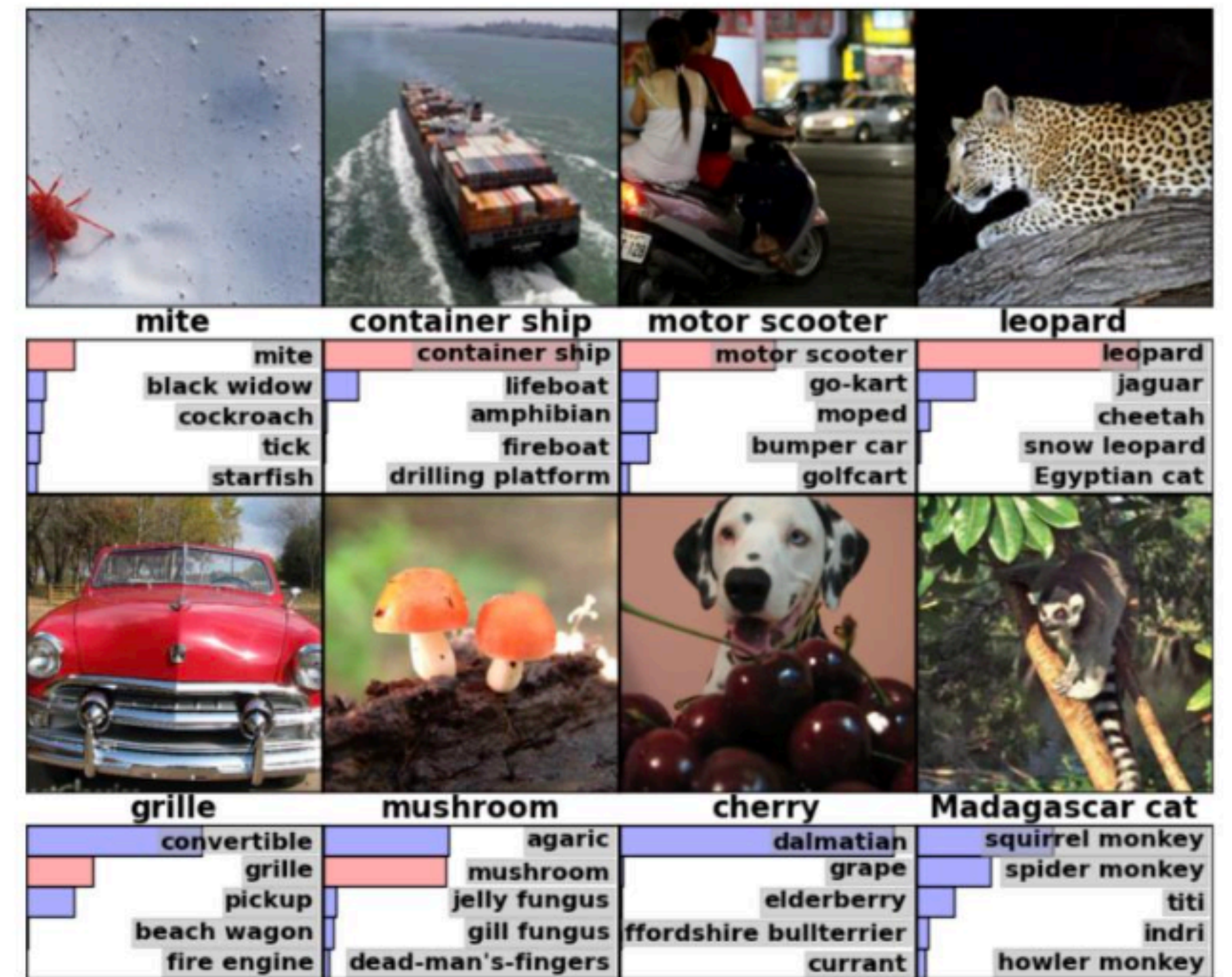
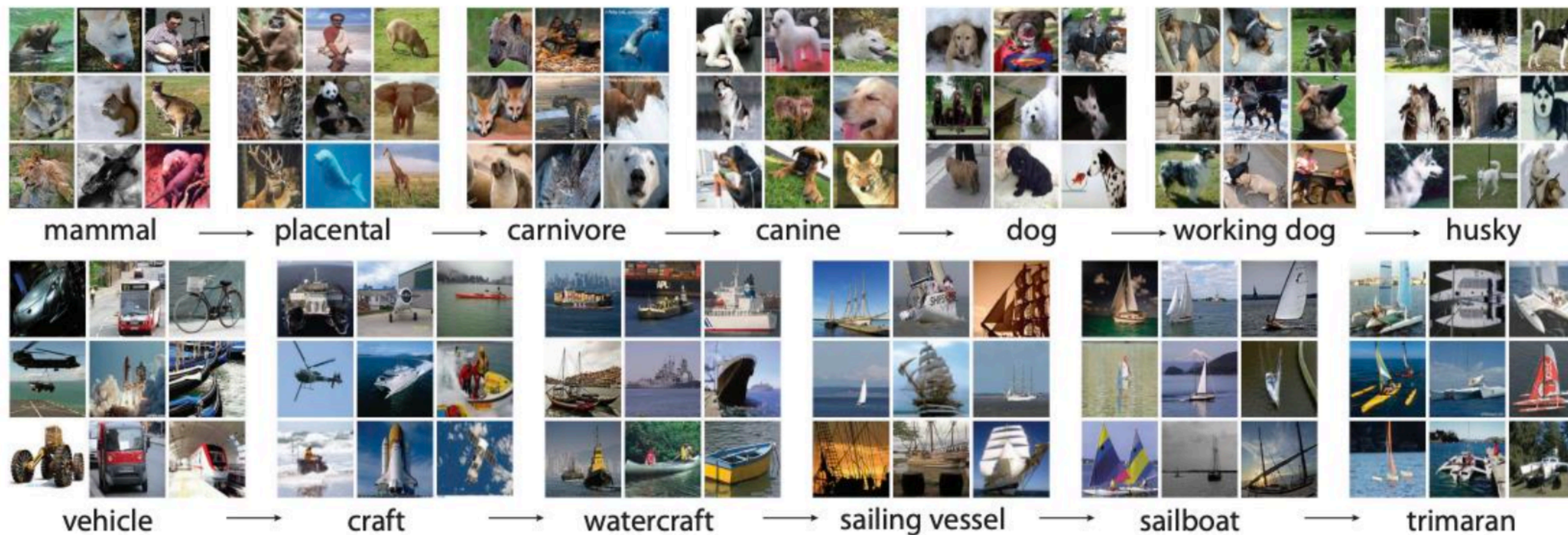
Diffusion on MNIST



Part 2: Visual Perception

Visual Perception

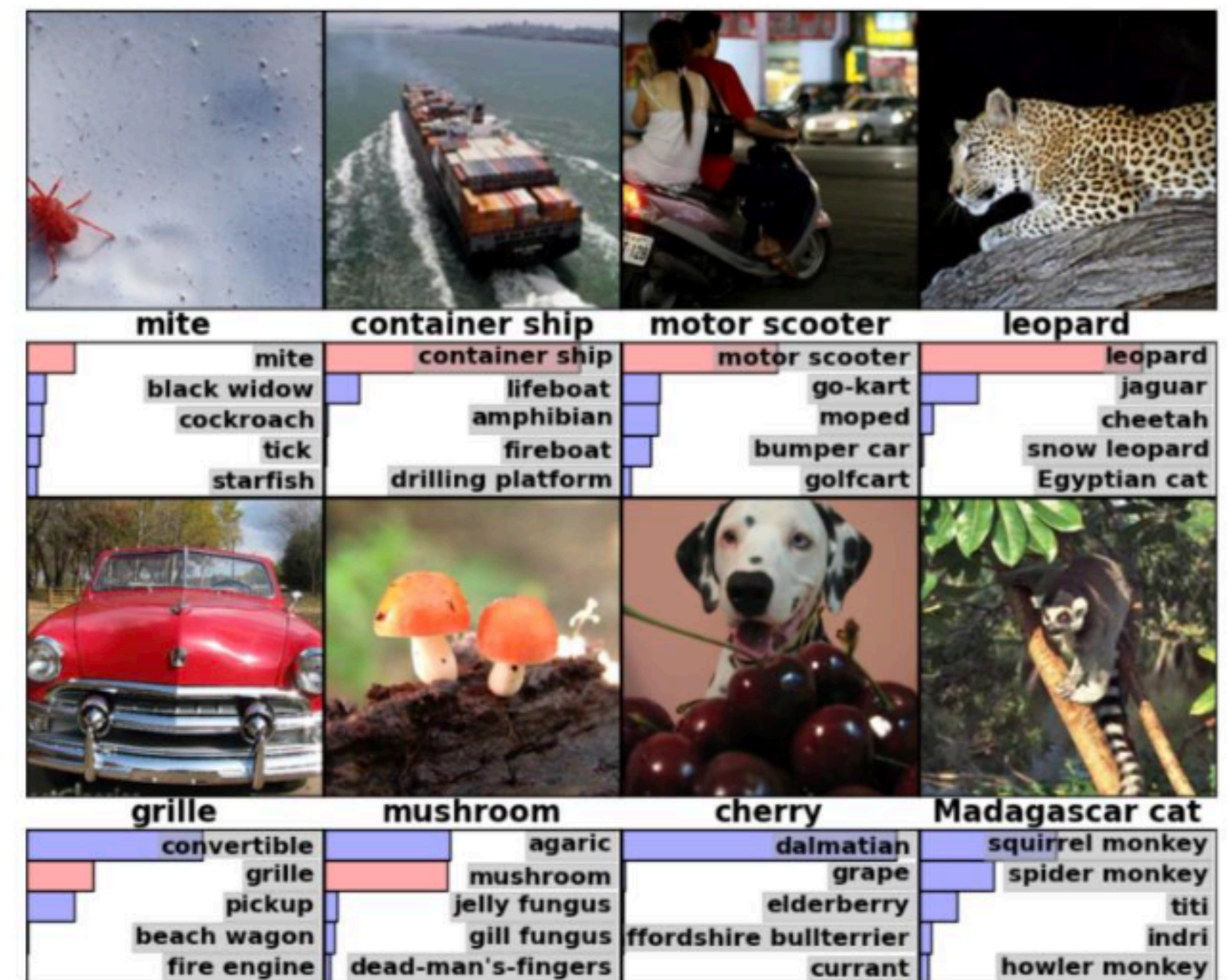
ImageNet Dataset / Classification



Visual Perception

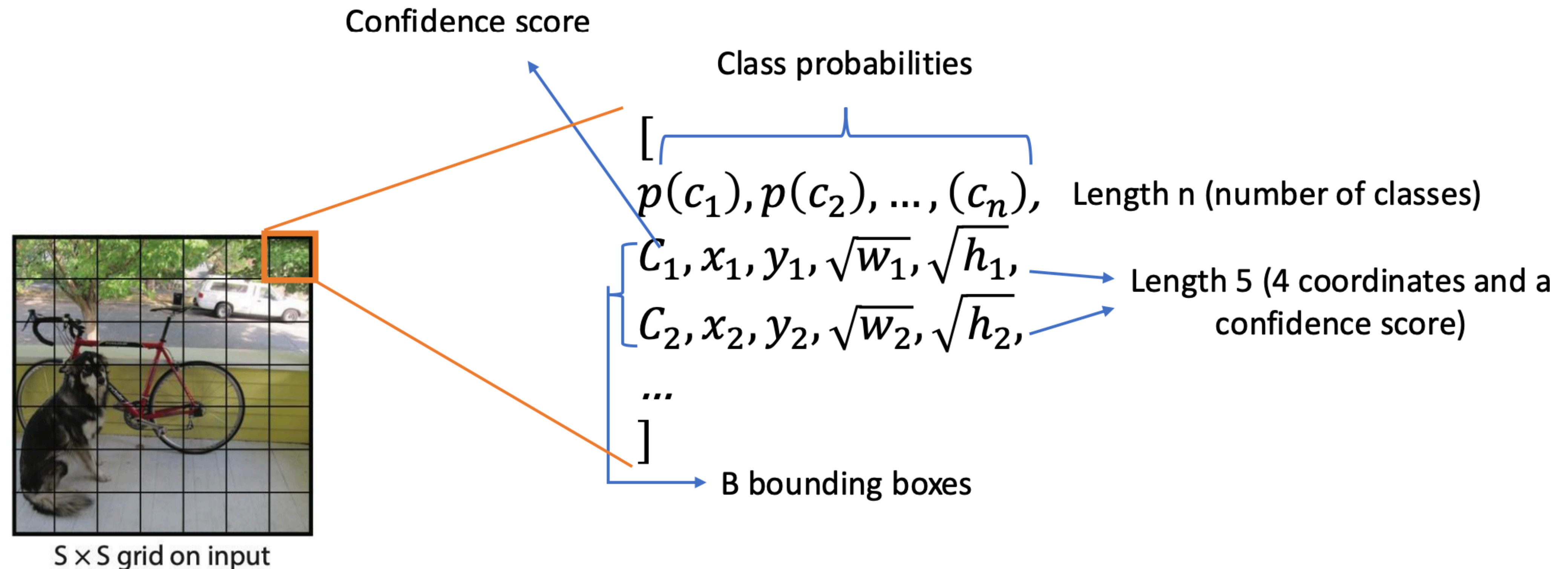
ImageNet Dataset / Classification

- At the 2012 ImageNet Large Scale Visual Recognition Challenge, AlexNet achieved a top-5 error rate of 15.3%, outperforming the runner-up by a significant margin of 10.8%.
- Since 2015, deep neural networks have surpassed human performance on this task.



Visual Perception

Object Detection

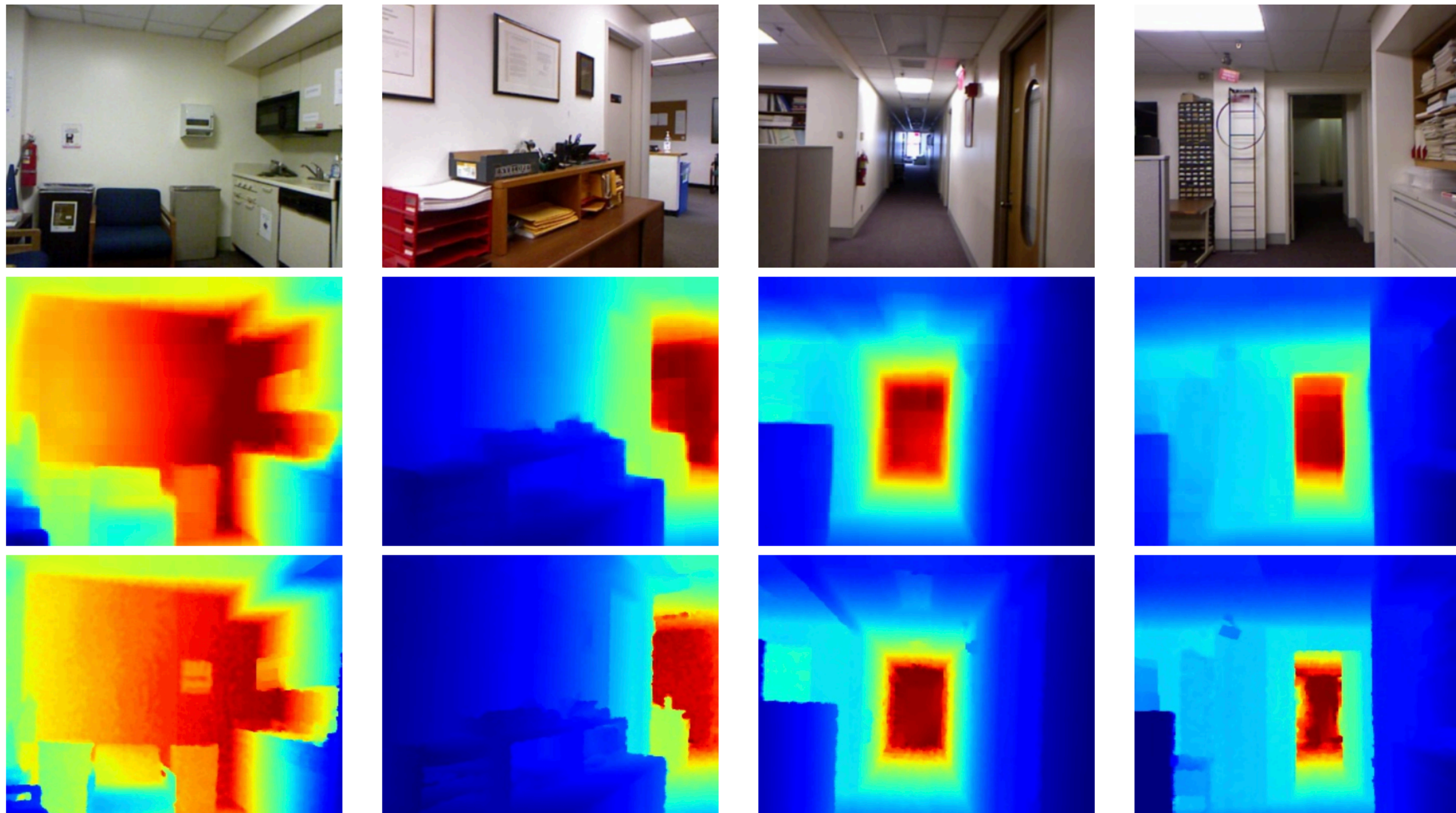


Overall size: $(B \times 5 + n)$

In the case of the original YOLOv1, $B=2$ and $n=20$

Visual Perception

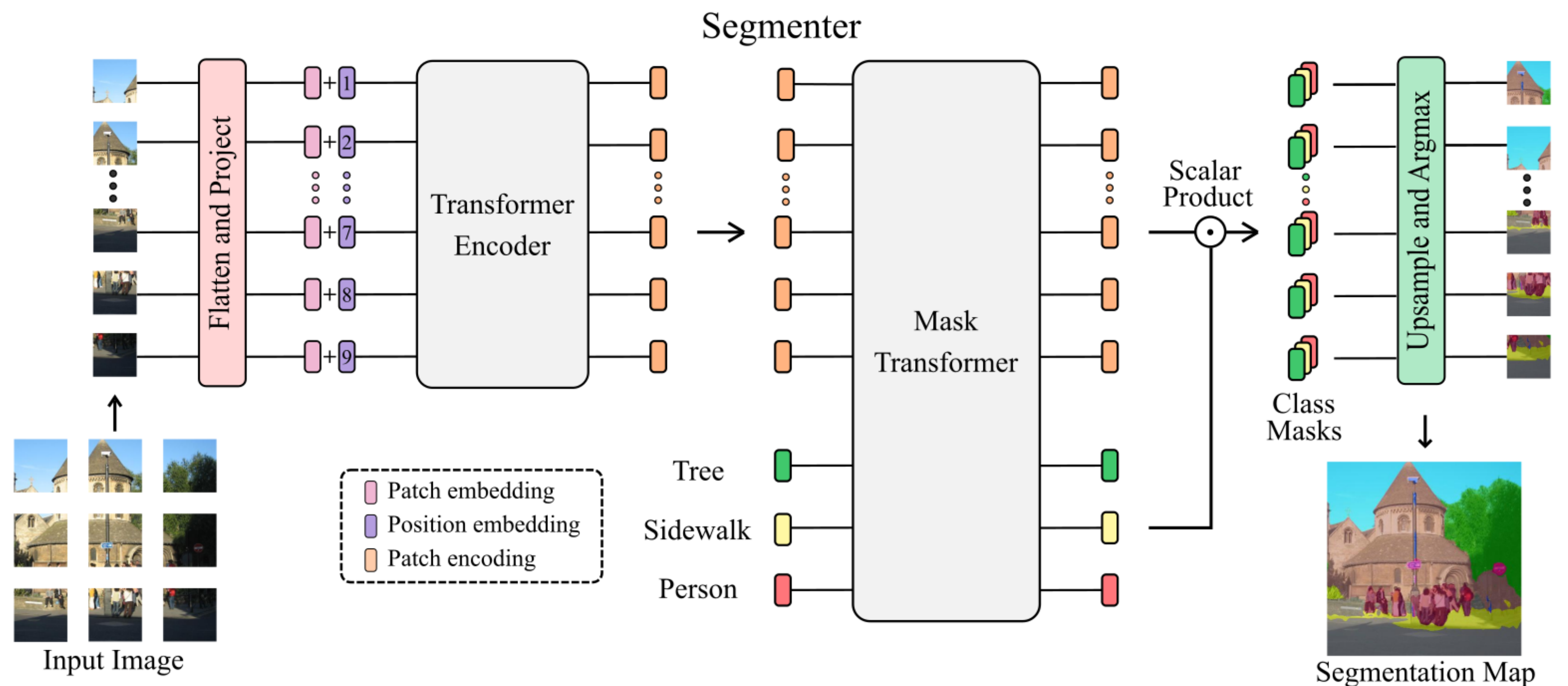
Depth Estimation



Visual Perception

Transformer-based Semantic Segmentation

- Break up the images into square patches
- Transform each patch into a feature vector
- Feed to a transformer architecture

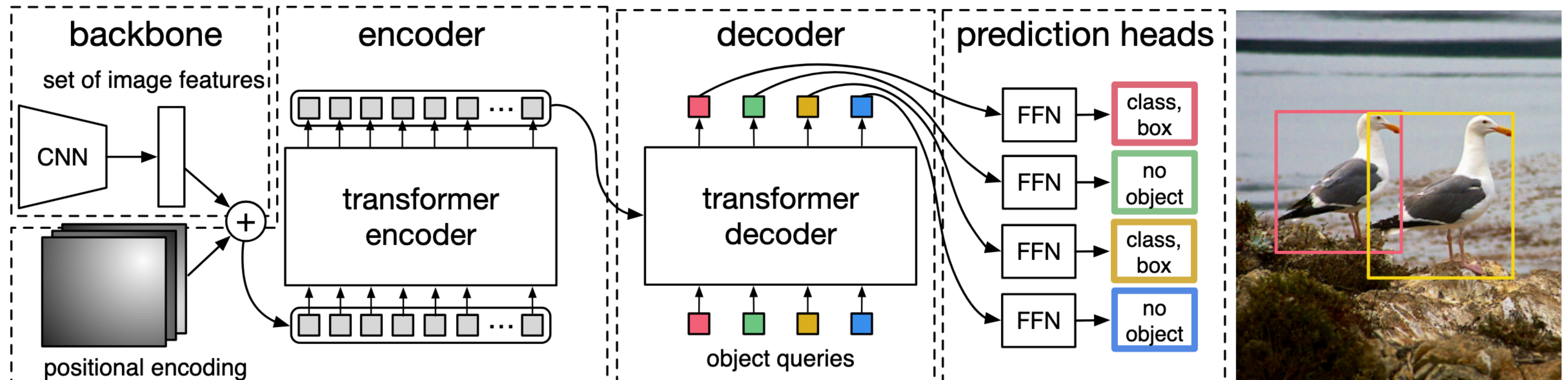


Visual Transformers Idea

Visual Perception

Transformer-based Object Detection

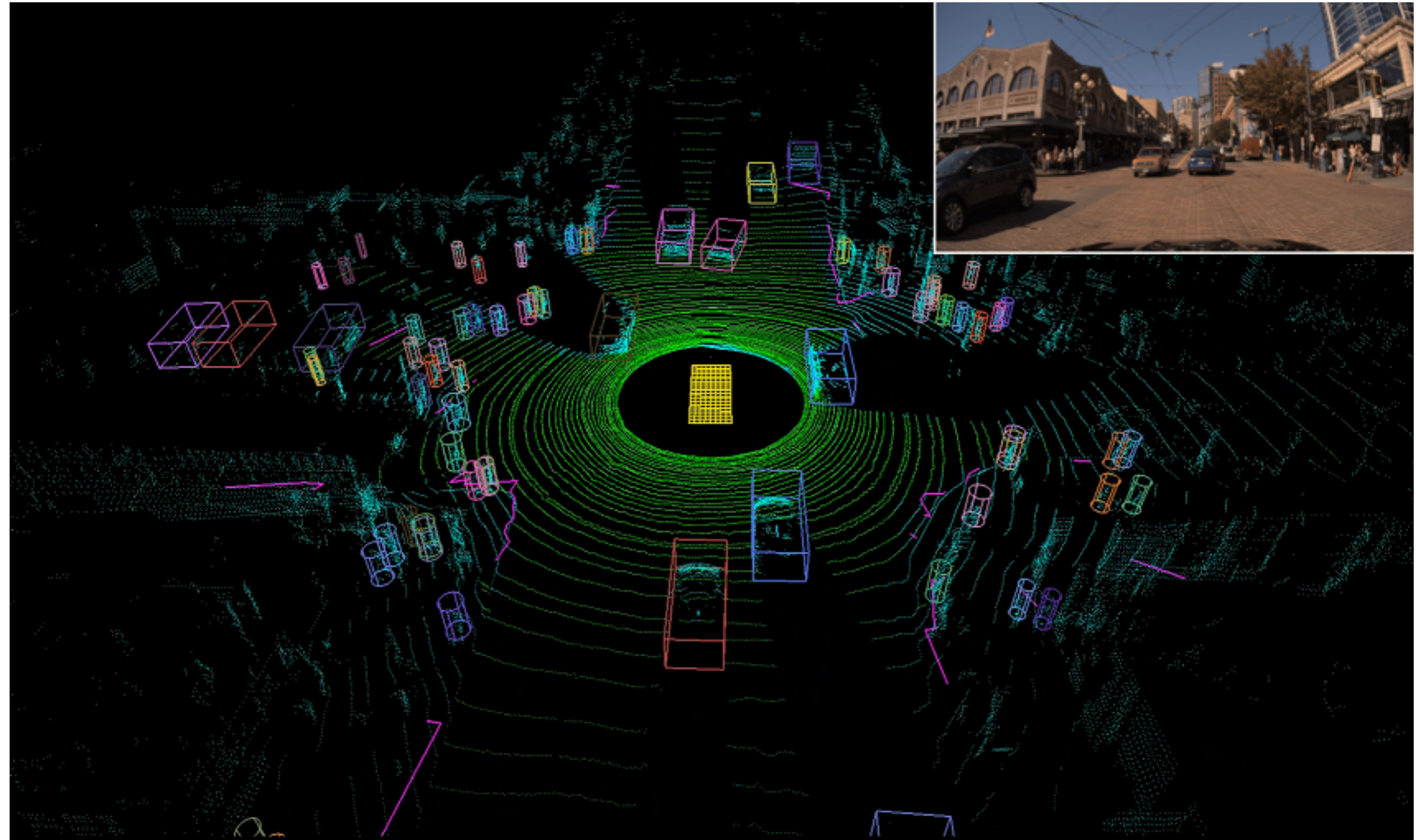
- Break up the images into square patches
- Each query passed through decoder is one predicted box



Visual Perception

Transformer-based 3d Object Detection

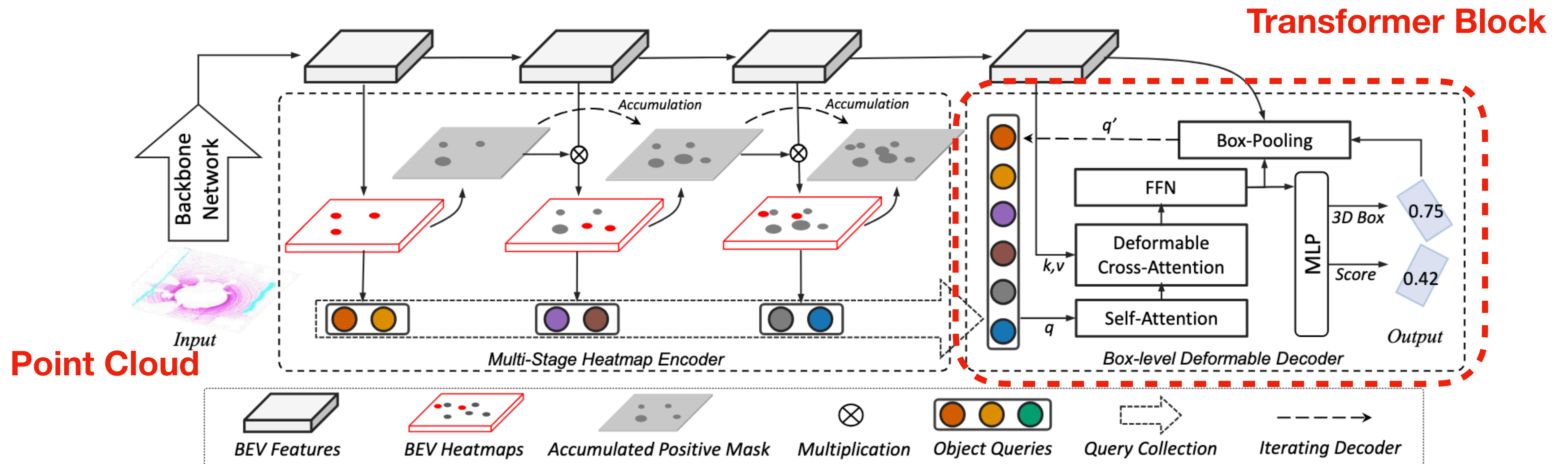
- 3d scene consists of N 3-dimensional points
- The task is to localize 3d objects and assign a 3d bounding box to each of them



Visual Perception

Transformer-based 3d Object Detection

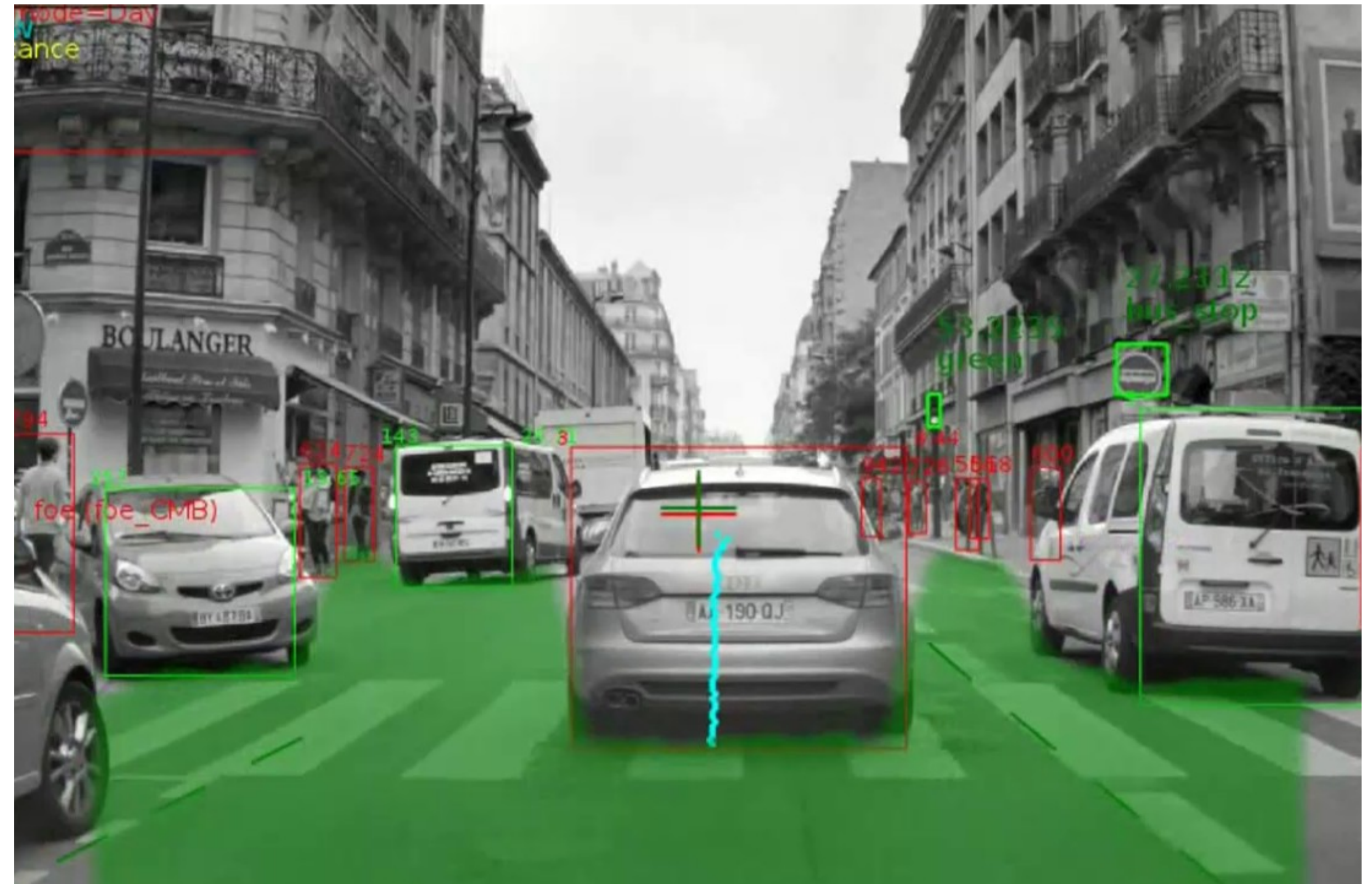
- “Flatten” 3d point cloud and generate 2d scene representation (BEV)
- Extract queries and each query passed through the decoder is one predicted box



Visual Perception

Practical 3d Object Detection

- N thousands scenes collected and labelled in Paris
- All of the images were collected in sunny weather between 1 and 2 pm
- We split the collected data into train and test, train the model and observe good performance on test set



Visual Perception

Practical 3d Object Detection

- We try to make predictions for the following scene
- What performance should we expect?
- Why?



Visual Perception

Practical 3d Object Detection

- We try to make predictions for the following scene
- What performance should we expect?
- Why?



Visual Perception

Practical 3d Object Detection

- We try to make predictions for the following scene
- What performance should we expect?
- Why?



Somewhere in Los Angeles

Visual Perception

Practical 3d Object Detection

- When we train the model on one type of data and test it on a different one, we consistently observe a decline in performance.
- This is a common phenomenon when dealing with out-of-distribution (OOD) data.

Setting	Source\Target	KITTI	Argoverse	nuScenes	Lyft	Waymo
Easy	KITTI	88.0 / 82.5	55.8 / 27.7	47.4 / 13.3	81.7 / 51.8	45.2 / 11.9
	Argoverse	69.5 / 33.9	79.2 / 57.8	52.5 / 21.8	86.9 / 67.4	83.8 / 40.2
	nuScenes	49.7 / 13.4	73.2 / 21.8	73.4 / 38.1	89.0 / 38.2	78.8 / 36.7
	Lyft	74.3 / 39.4	77.1 / 45.8	63.5 / 23.9	90.2 / 87.3	87.0 / 64.7
	Waymo	51.9 / 13.1	76.4 / 42.6	55.5 / 21.6	87.9 / 74.5	90.1 / 85.3
Moderate	KITTI	80.6 / 68.9	44.9 / 22.3	26.2 / 8.3	61.8 / 33.7	43.9 / 12.3
	Argoverse	56.6 / 31.4	69.9 / 44.2	27.6 / 11.8	66.6 / 42.1	72.3 / 35.1
	nuScenes	39.8 / 10.7	56.6 / 17.1	40.7 / 21.2	71.4 / 25.0	68.2 / 30.8
	Lyft	61.1 / 34.3	62.5 / 35.3	33.6 / 12.3	83.7 / 65.5	77.6 / 53.2
	Waymo	45.8 / 13.2	64.4 / 29.8	28.9 / 13.7	74.2 / 53.8	85.9 / 67.9
Hard	KITTI	81.9 / 66.7	42.5 / 22.2	24.9 / 8.8	57.4 / 34.2	41.5 / 12.6
	Argoverse	58.5 / 33.3	69.9 / 42.8	26.8 / 14.5	64.4 / 42.7	68.5 / 36.8
	nuScenes	39.6 / 10.1	53.3 / 16.7	40.2 / 20.5	67.7 / 25.7	66.9 / 29.0
	Lyft	60.7 / 33.9	62.9 / 35.9	30.6 / 11.7	79.3 / 65.5	77.0 / 53.9
	Waymo	46.3 / 12.6	61.6 / 29.0	28.4 / 14.1	74.1 / 54.5	80.4 / 67.7

Bonus Questions

Asking ChatGPT

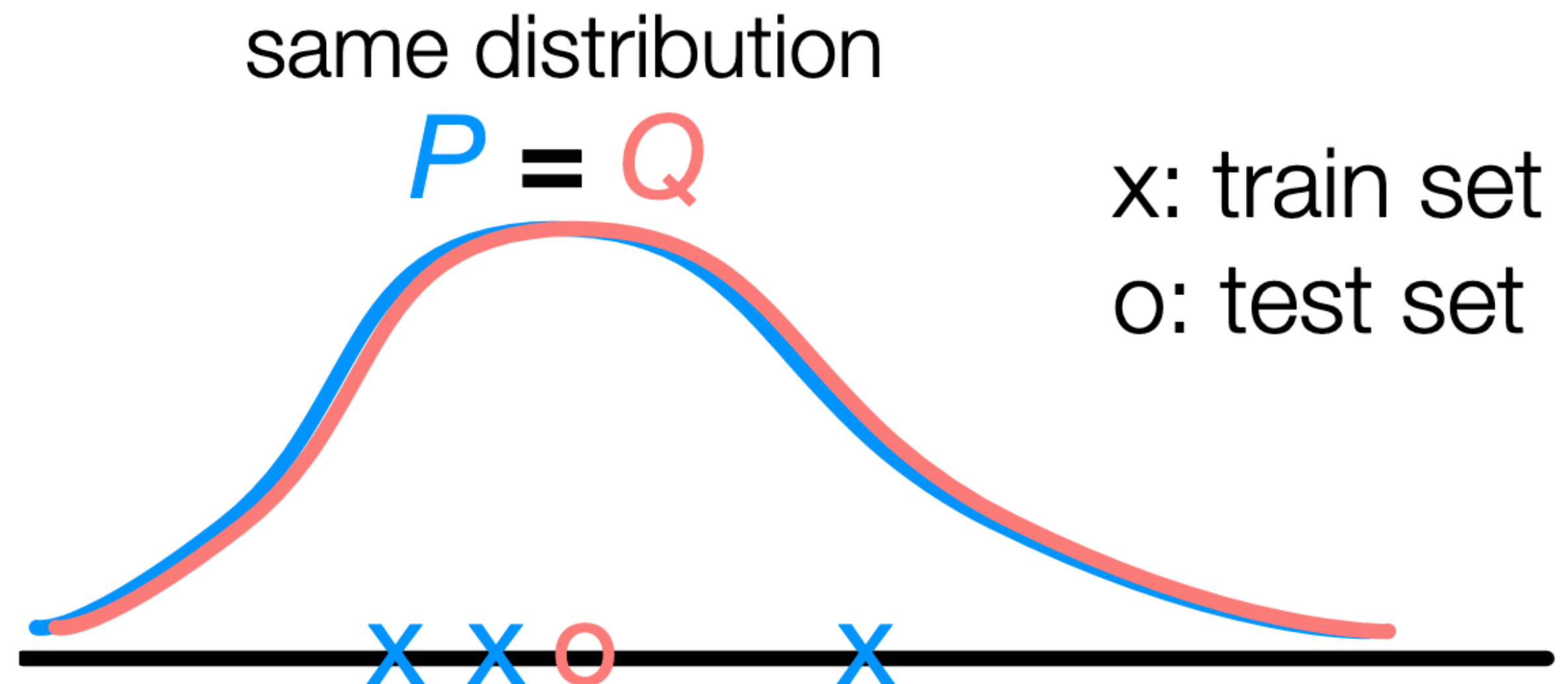
Why ChatGPT can seemingly answer any question?

Part 3: Generalization

Generalization

General Terms

- **In theory:** same distribution for training and testing



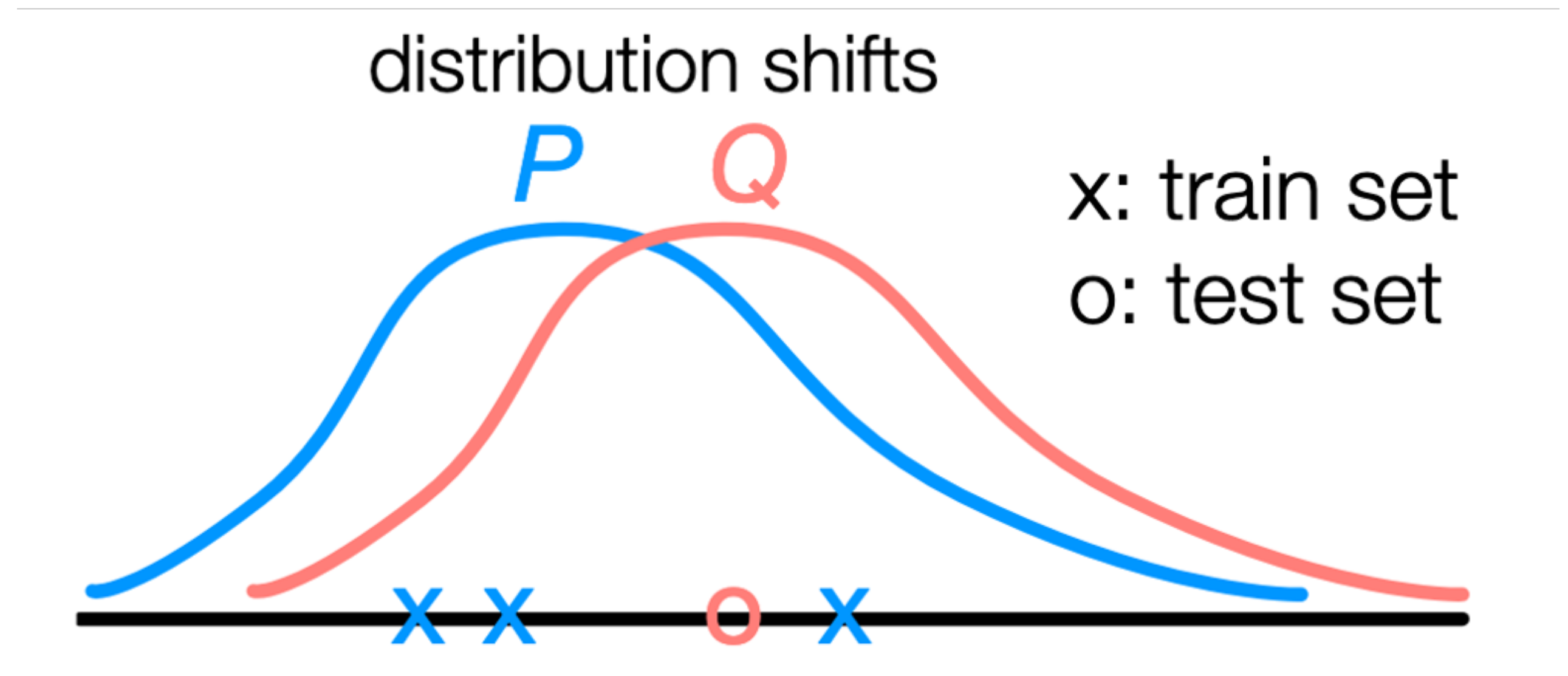
Generalization

General Terms

In theory: same distribution for training and testing

In the real word: distribution shifts are everywhere

Especially in cases of Bayesian optimization and active learning!

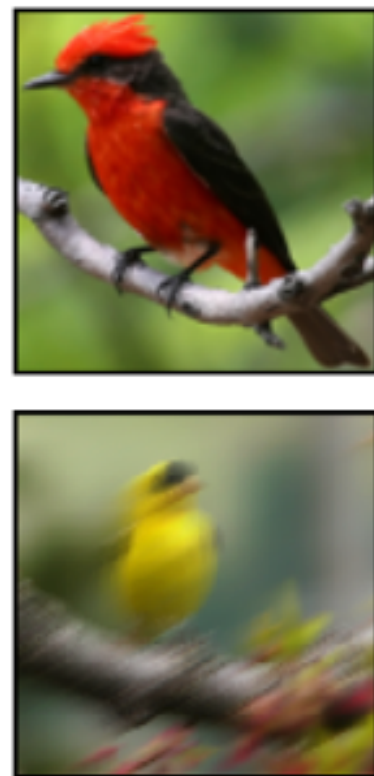


Generalization

General Terms

In theory: same distribution for training and testing

In the real word: distribution shifts are everywhere



Hendrycks and Dietterich, 2018

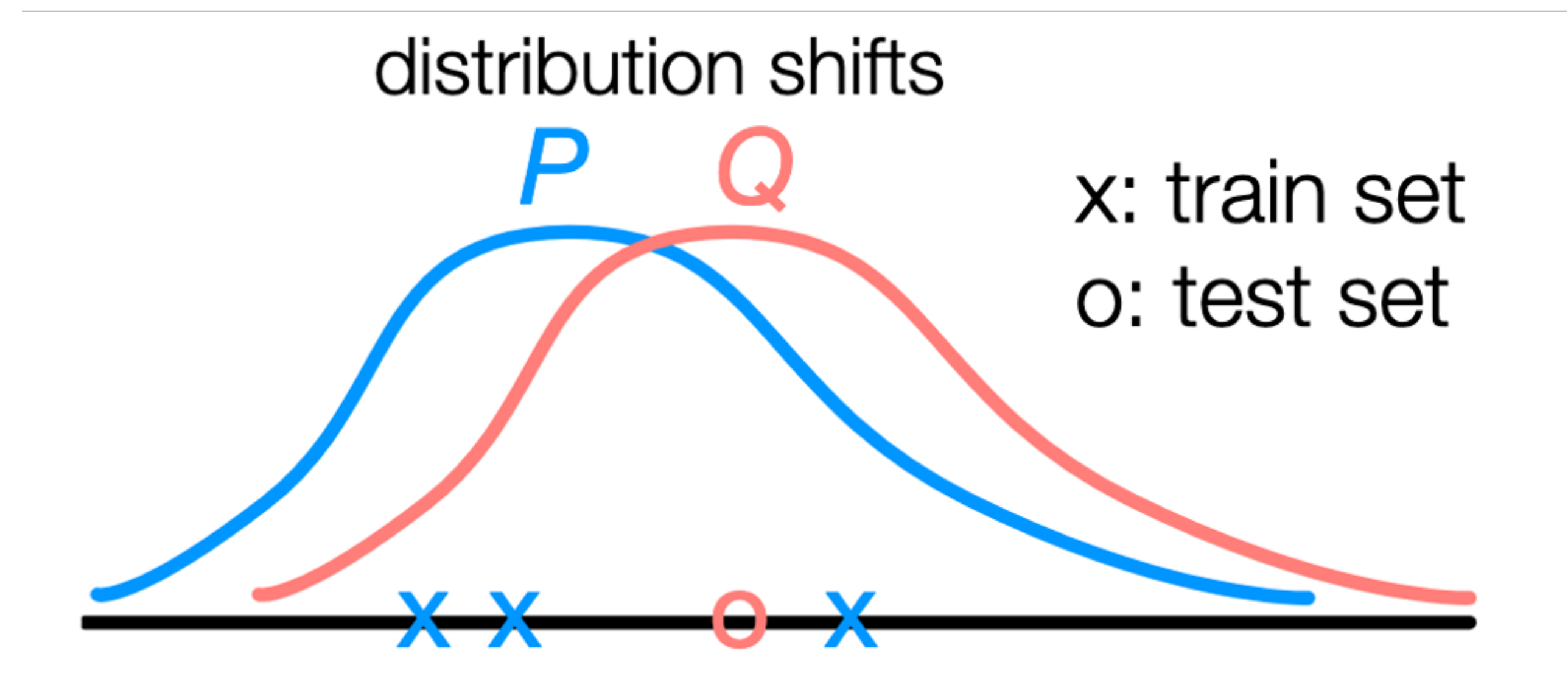


Recht, Roelofs, Schmidt and Shankar, 2019

Generalization

General Terms

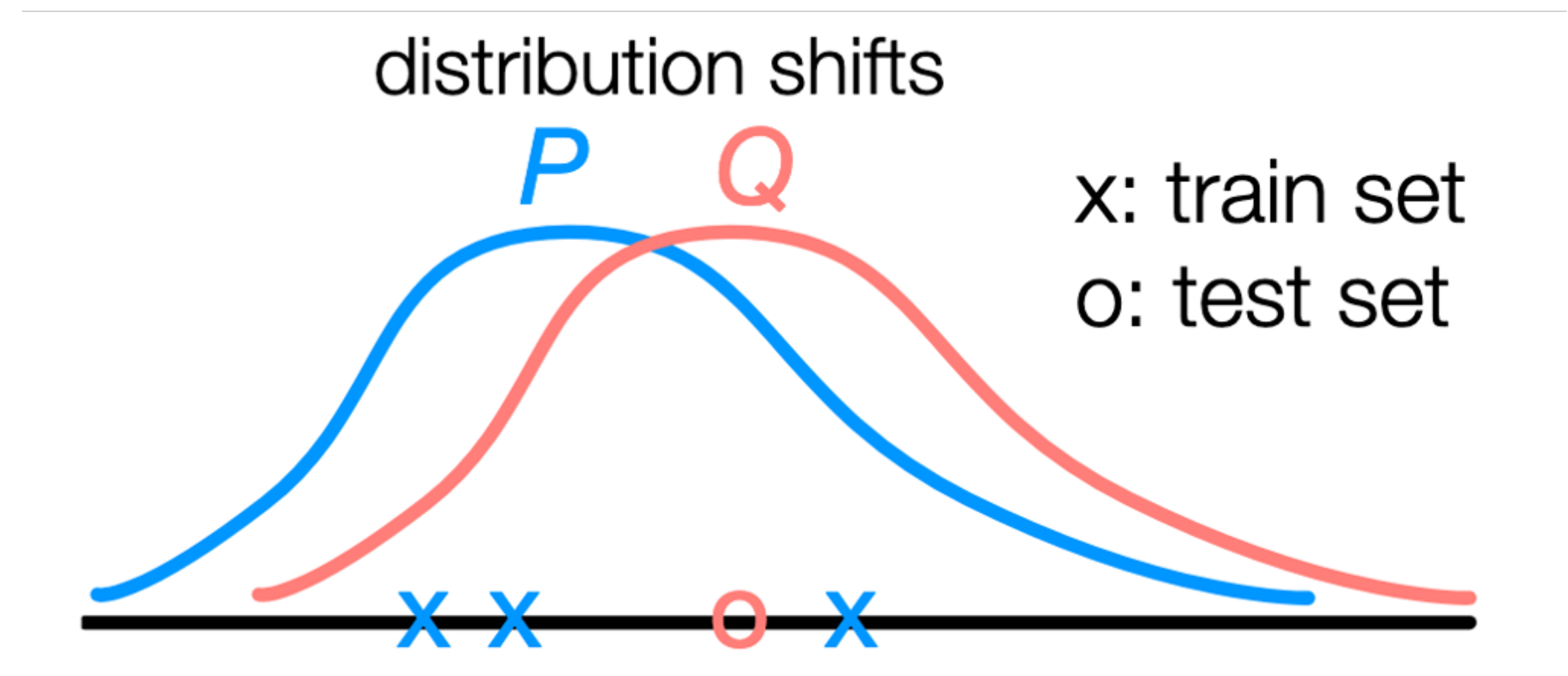
- **Domain adaptation**
 - Data from the test distribution (may be unlabeled)



Generalization

General Terms

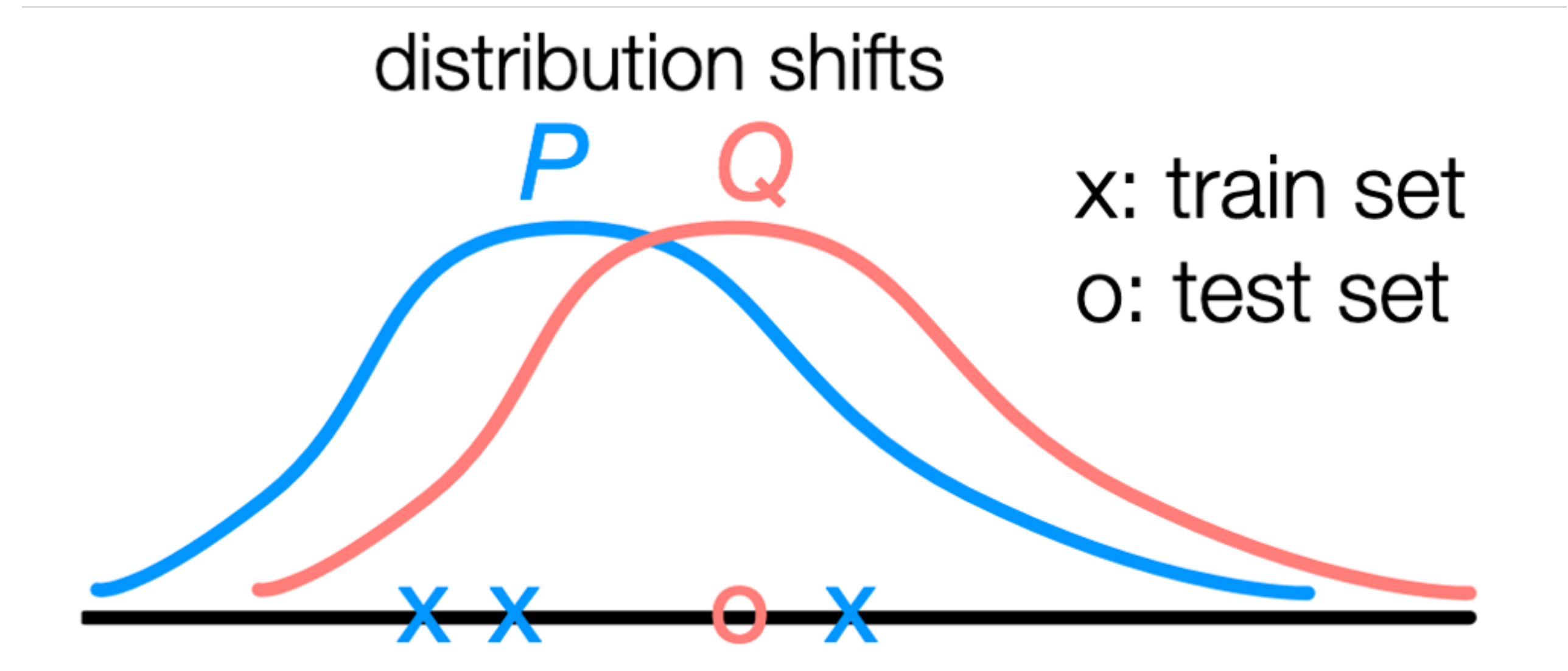
- **Domain adaptation**
 - Data from the test distribution (may be unlabeled)
 - Hard to know the test distribution



Generalization

General Terms

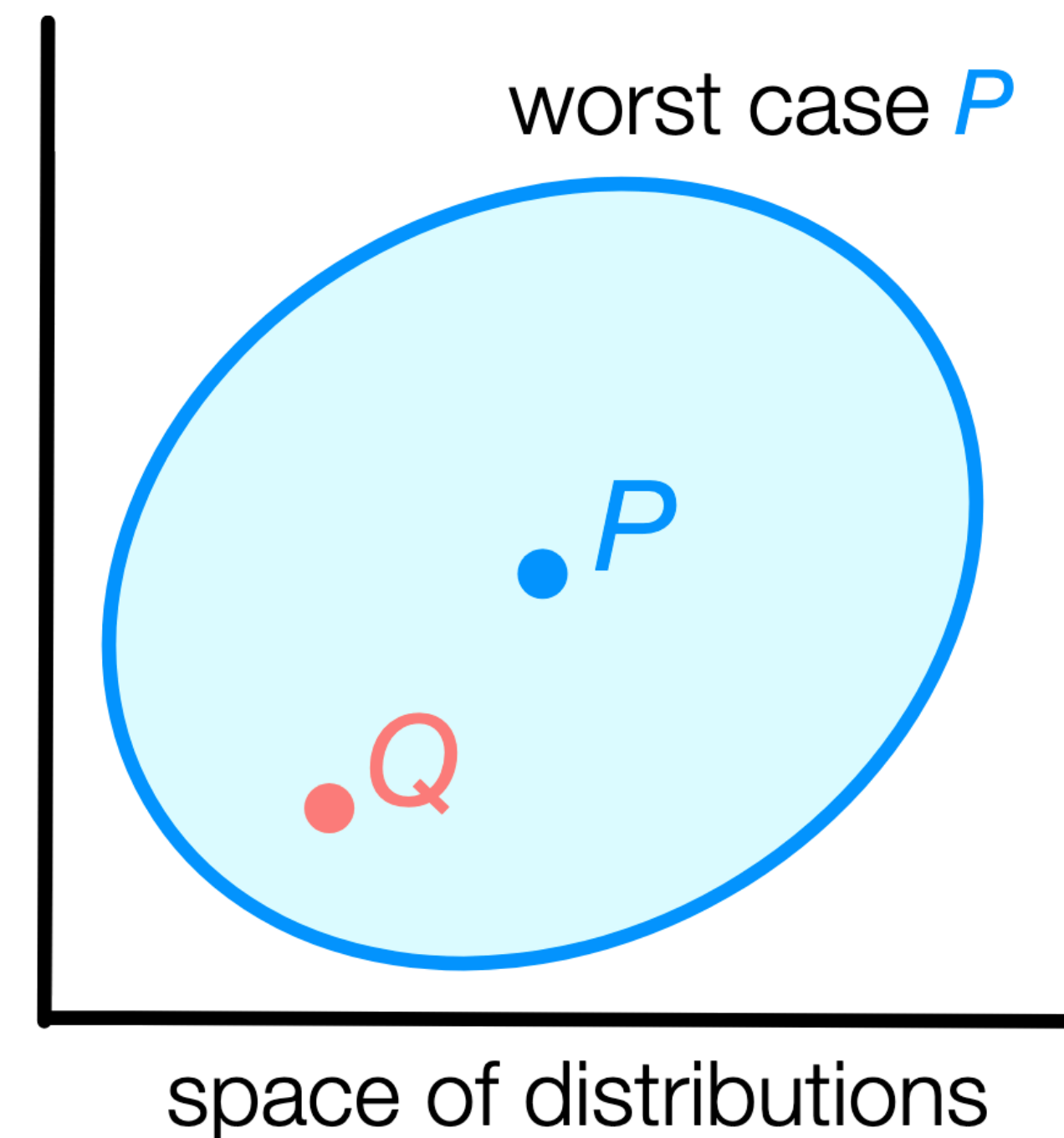
- **Domain adaptation**
 - Data from the test distribution (may be unlabeled)
 - Hard to know the test distribution
- **Domain generalization**
 - Data from the meta distribution



Generalization

General Terms

- **Domain adaptation**
 - Data from the test distribution (may be unlabeled)
 - Hard to know the test distribution
- **Domain generalization**
 - Data from the meta distribution
 - Hard to know the meta distribution
- **Adversarial robustness**



Generalization

Test-Time Training

In short: Test-time training approach computes a self-supervised loss (using only input) and optimizes the model for this particular samples

Final prediction is made using new updated weights

$$\theta^* = \theta - \eta \nabla_{\theta} \mathcal{L}_{self}(\mathbf{x}_{test}, \theta)$$

Where:

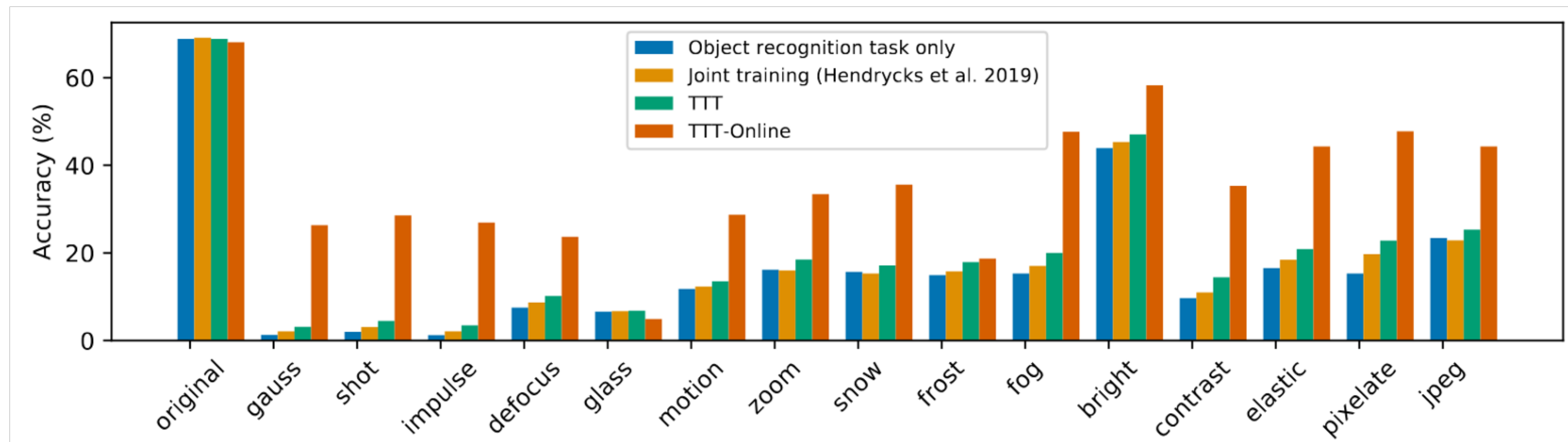
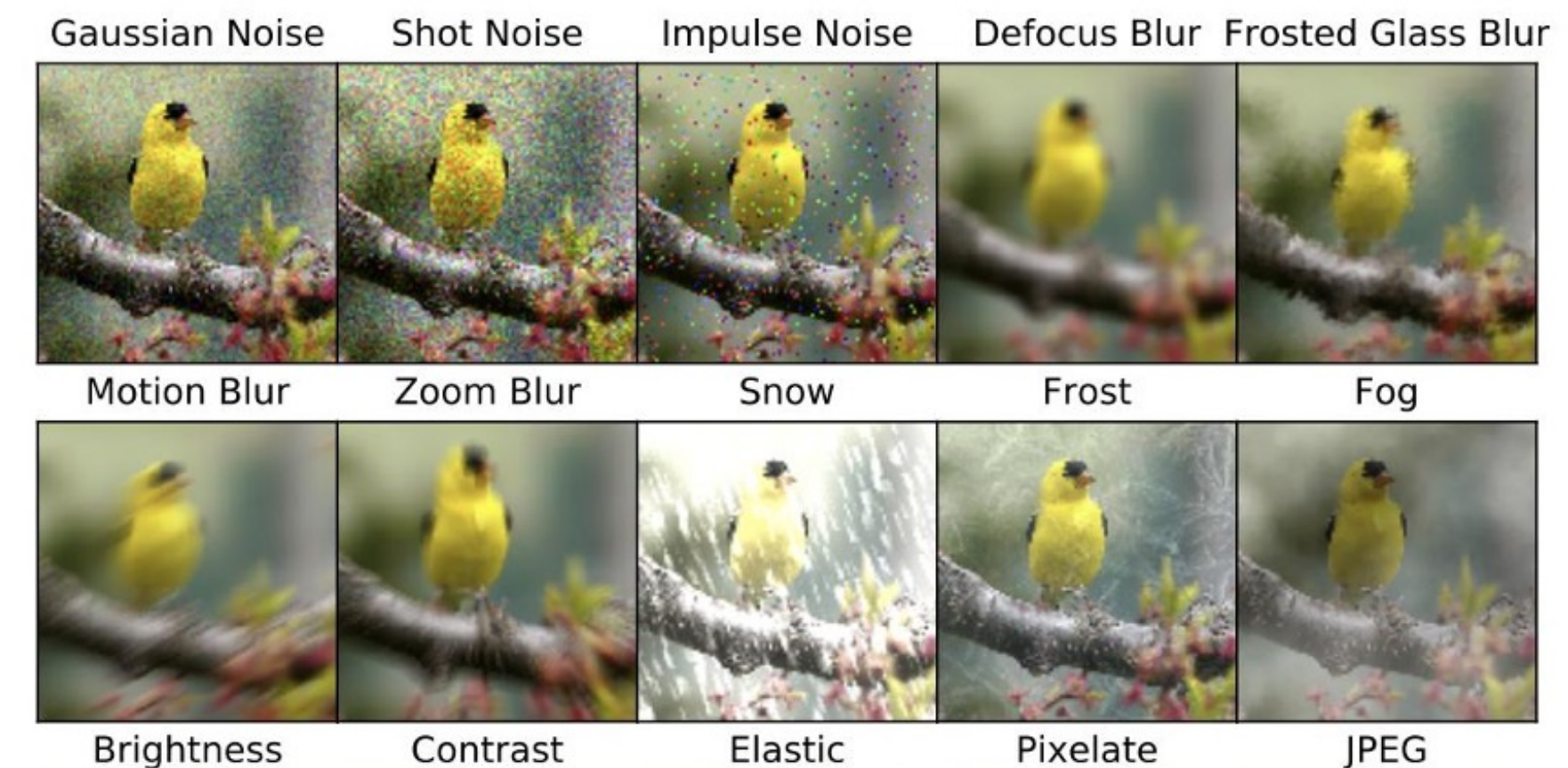
- θ = original model weights
- θ^* = updated parameters after TTT
- η = learning rate
- $\mathcal{L}_{self}$ = self-supervised loss (e.g., contrastive loss)
- $\mathbf{x}_{test}$ = test-time sample

Generalization

Test-Time Training

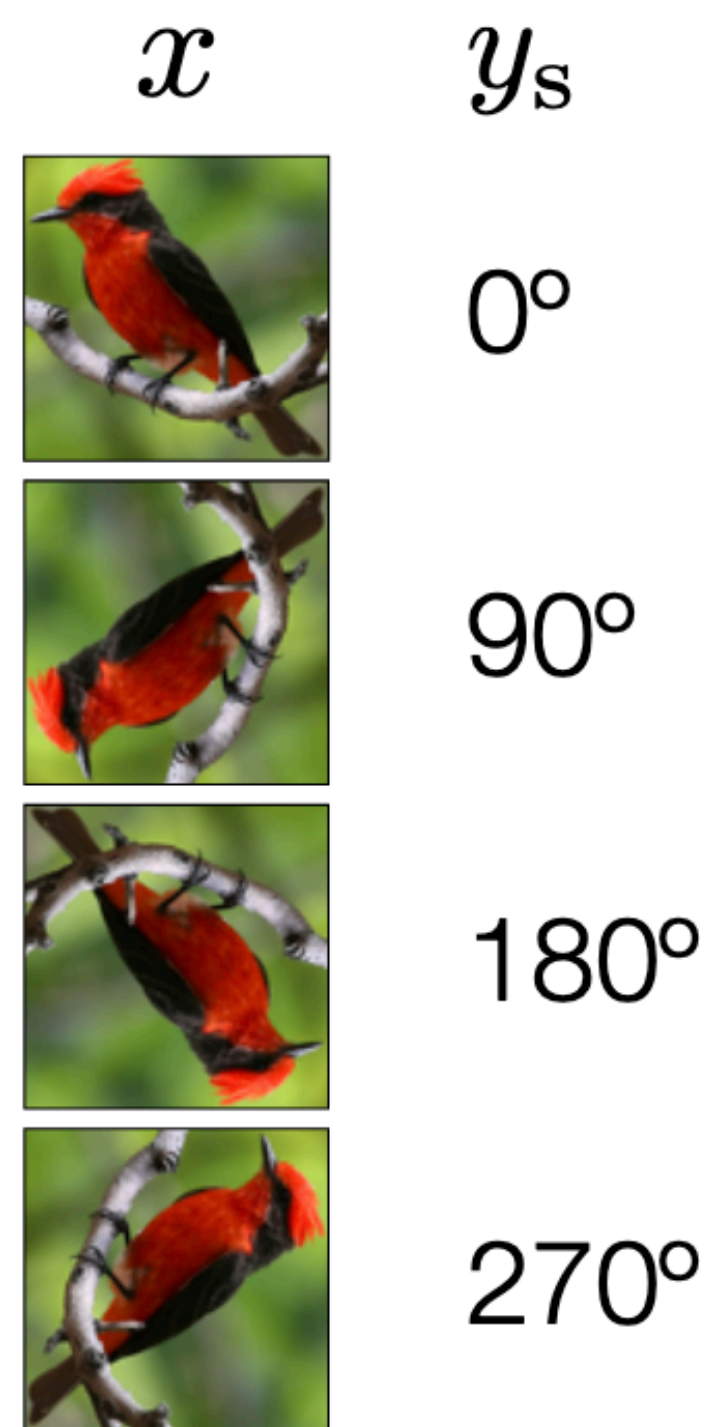
Though conceptually simple, TTT can lead to significant performance improvements on corrupted or out-of-distribution data.

Example: A model trained on ImageNet is then applied to the ImageNet test set with added corruptions. TTT demonstrates significant improvements in accuracy.



Test-Time Training (TTT)

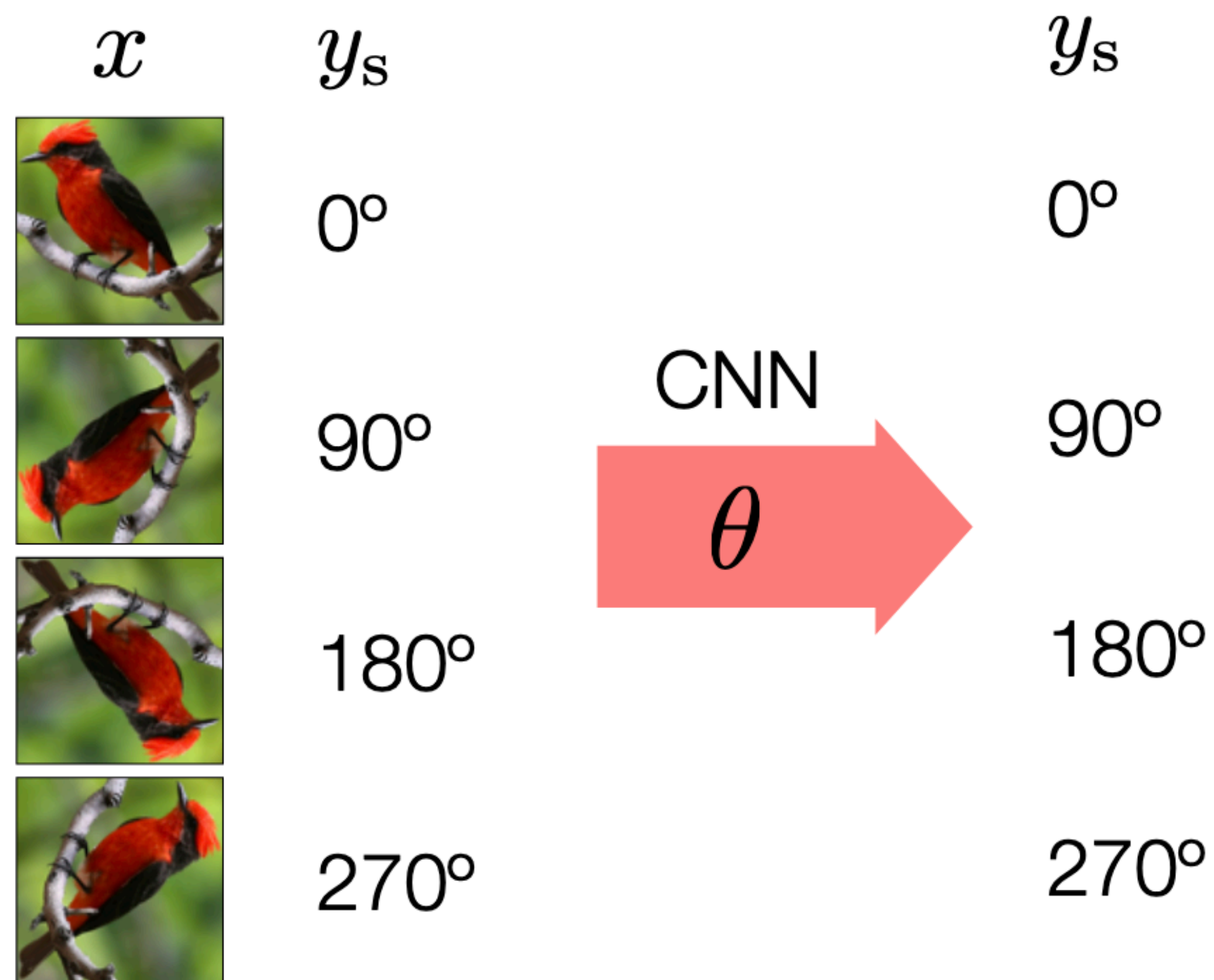
Rotation-based TTT



- Rotate input image by multiples of 90°

Test-Time Training

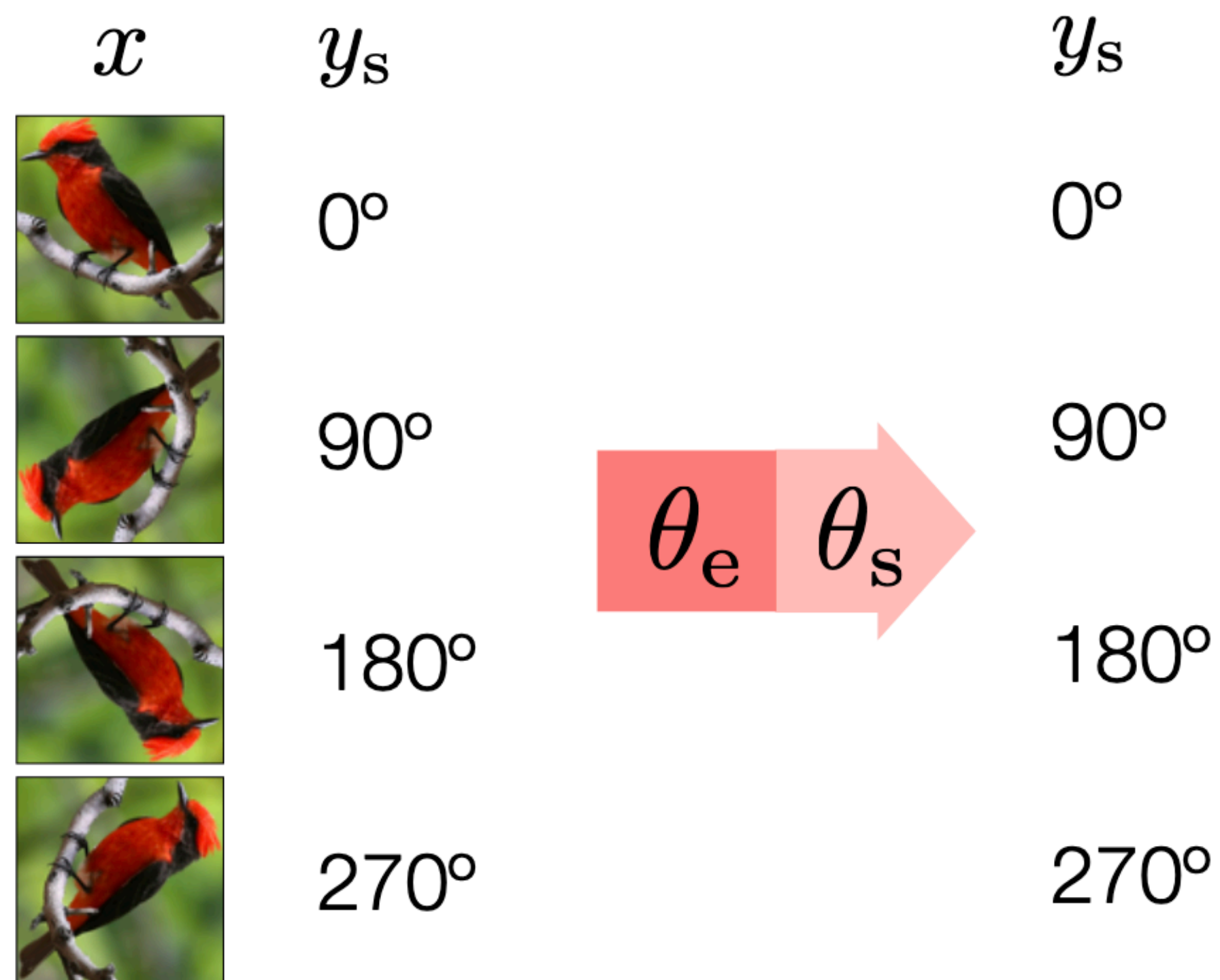
Rotation-based TTT



- Rotate input image by multiples of 90°
- Produce a four-way classification problem

Test-Time Training

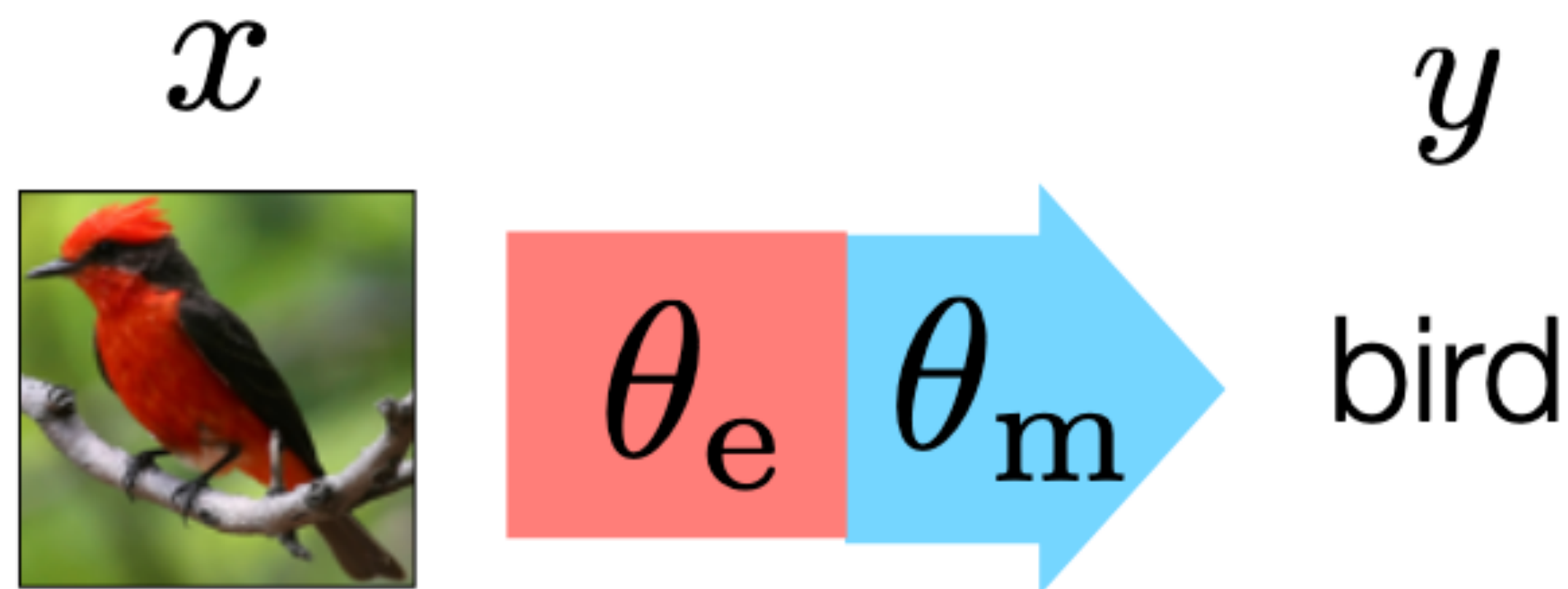
Rotation-based TTT



- Rotate input image by multiples of 90°
- Produce a four-way classification problem
- After training, take feature extractor

Test-Time Training

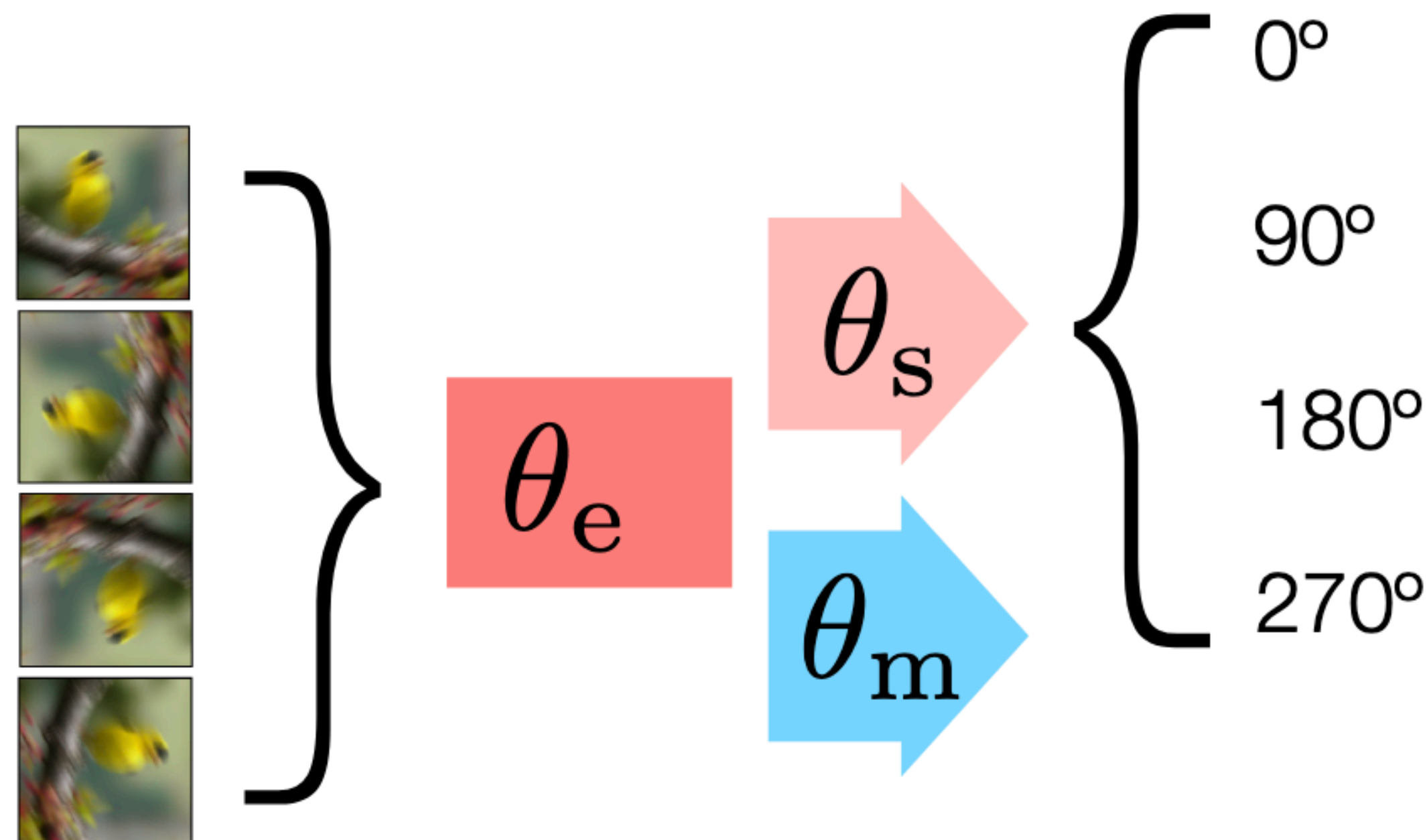
Rotation-based TTT



- Rotate input image by multiples of 90°
- Produce a four-way classification problem
- After training, take feature extractor
- Use it for a downstream main task

Test-Time Training

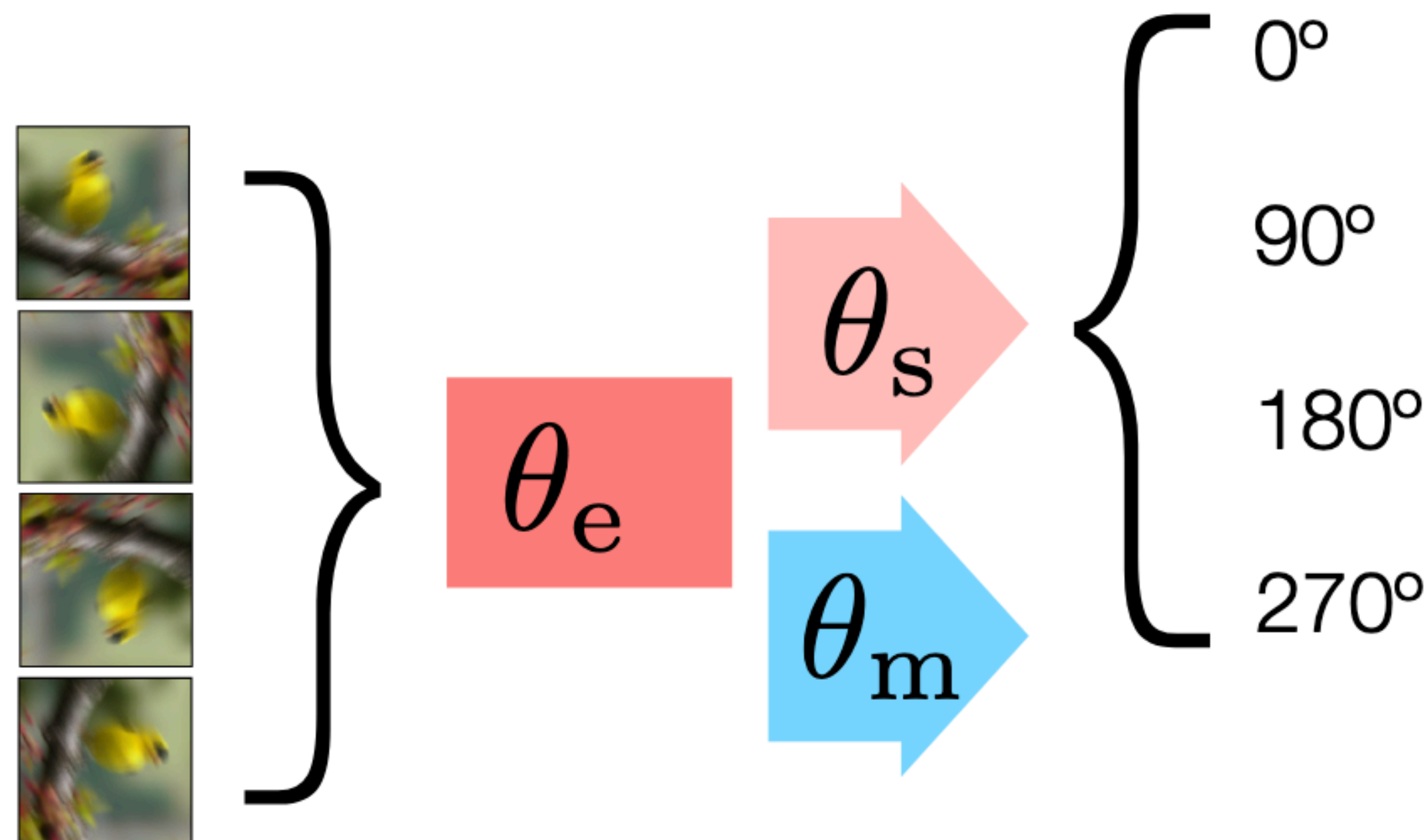
Rotation-based TTT



- During inference, we don't have access to image class, therefore, only rotation loss is available

Test-Time Training

Rotation-based TTT



- During inference, we don't have access to image class, therefore, only rotation loss is available
- Using the rotation loss as self-supervised loss in TTT, we optimize the model

Test-Time Training

TTT for MNIST

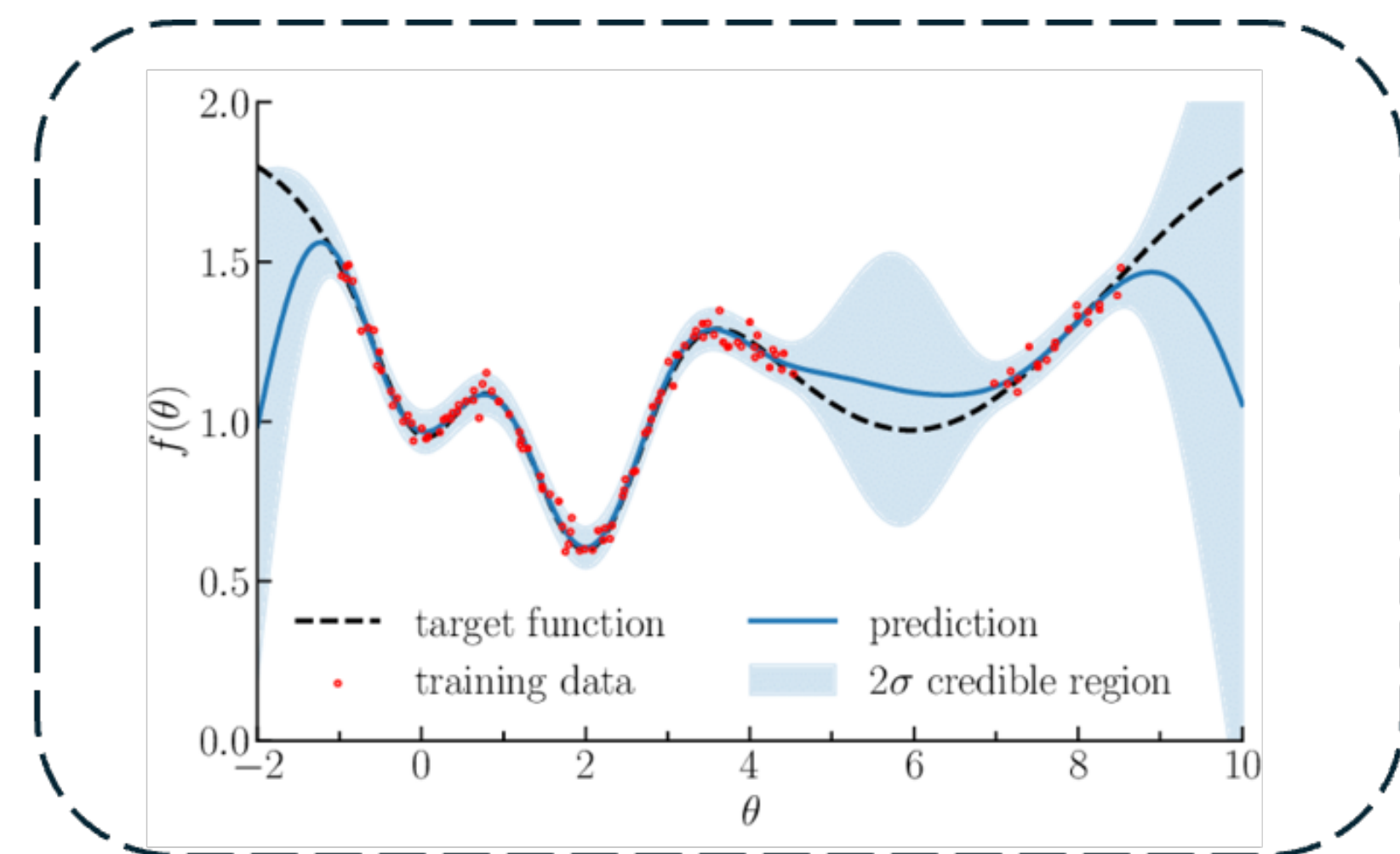
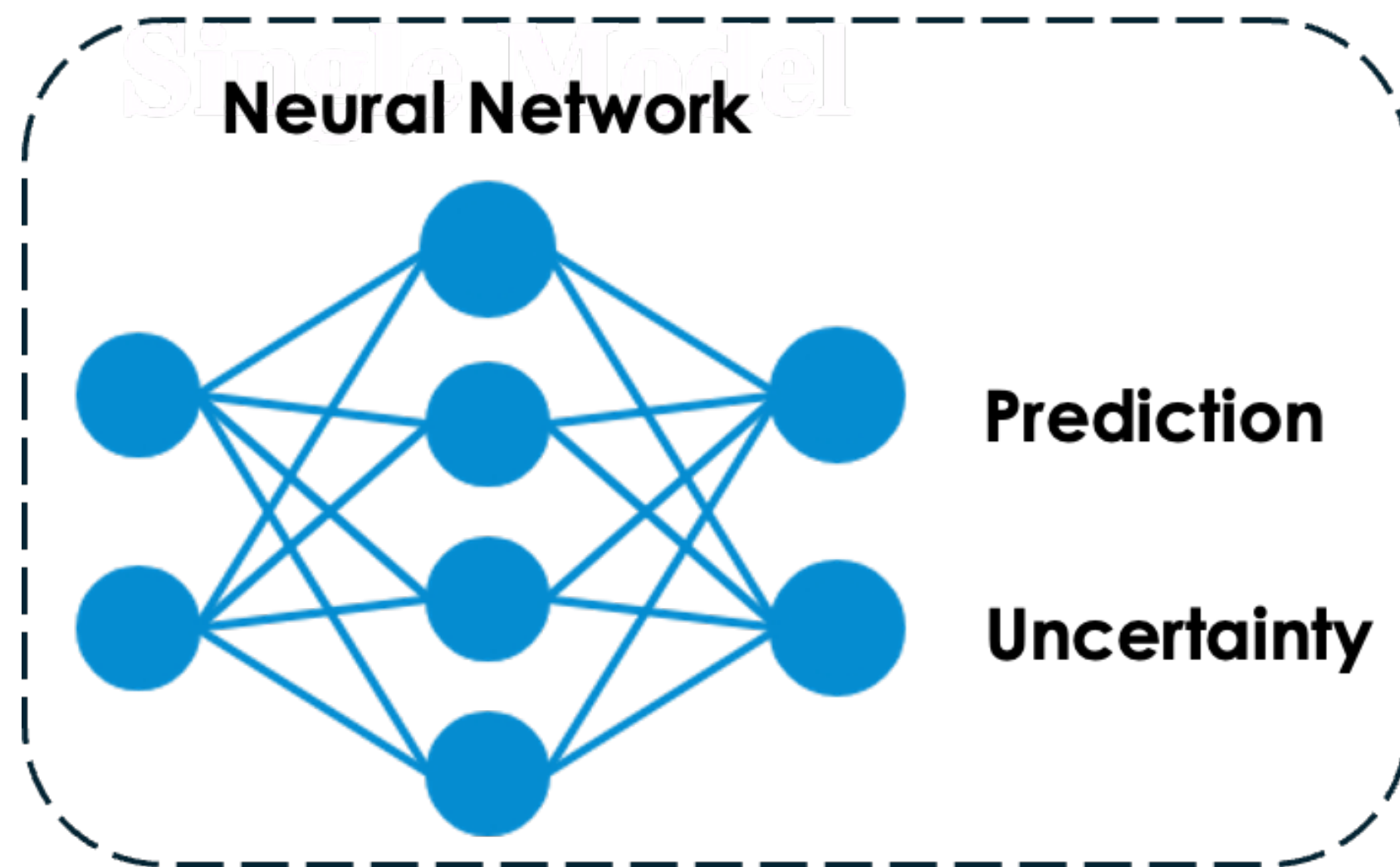


Part 4: Uncertainty Estimation

Uncertainty Estimation

Tasks Formulation

- Uncertainty Estimation is a long-standing problem in computer vision and machine learning
- Along with the predictions, we want to provide additional 'confidence' values.



Uncertainty Estimation

Uncertainty Applications

Uncertainty estimation is crucial for reliable decision-making, especially in critical applications.

In fields like:

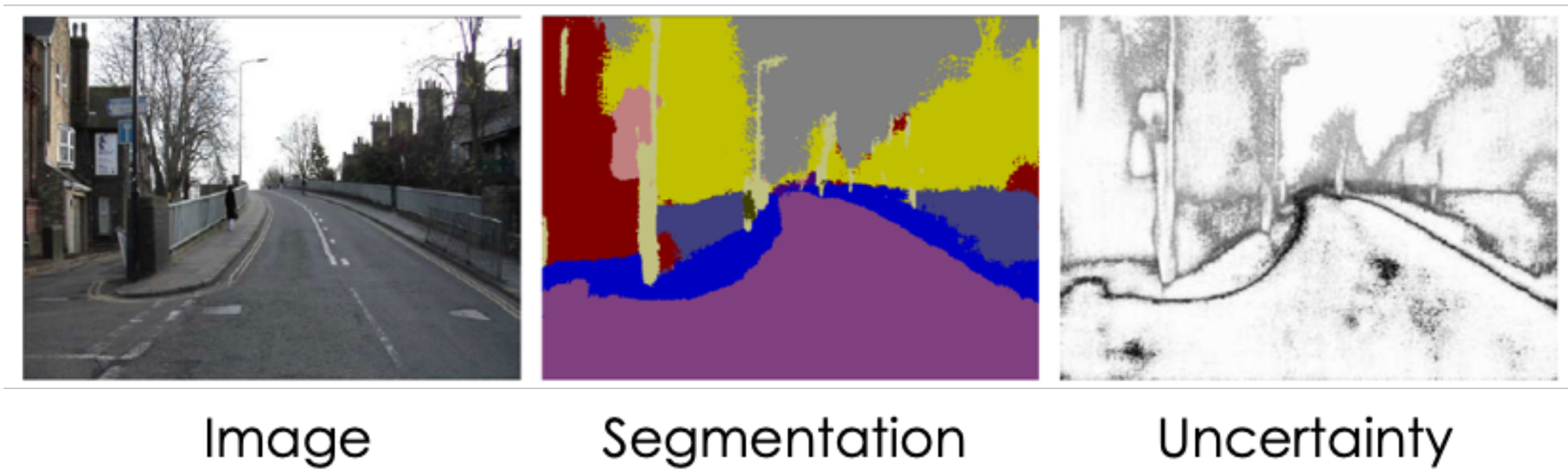
- 1) healthcare,**
- 2) autonomous driving,**
- 3) weather prediction, and**
- 4) financial forecasting,**

Understanding the level of certainty in model predictions can be as important as the predictions themselves.

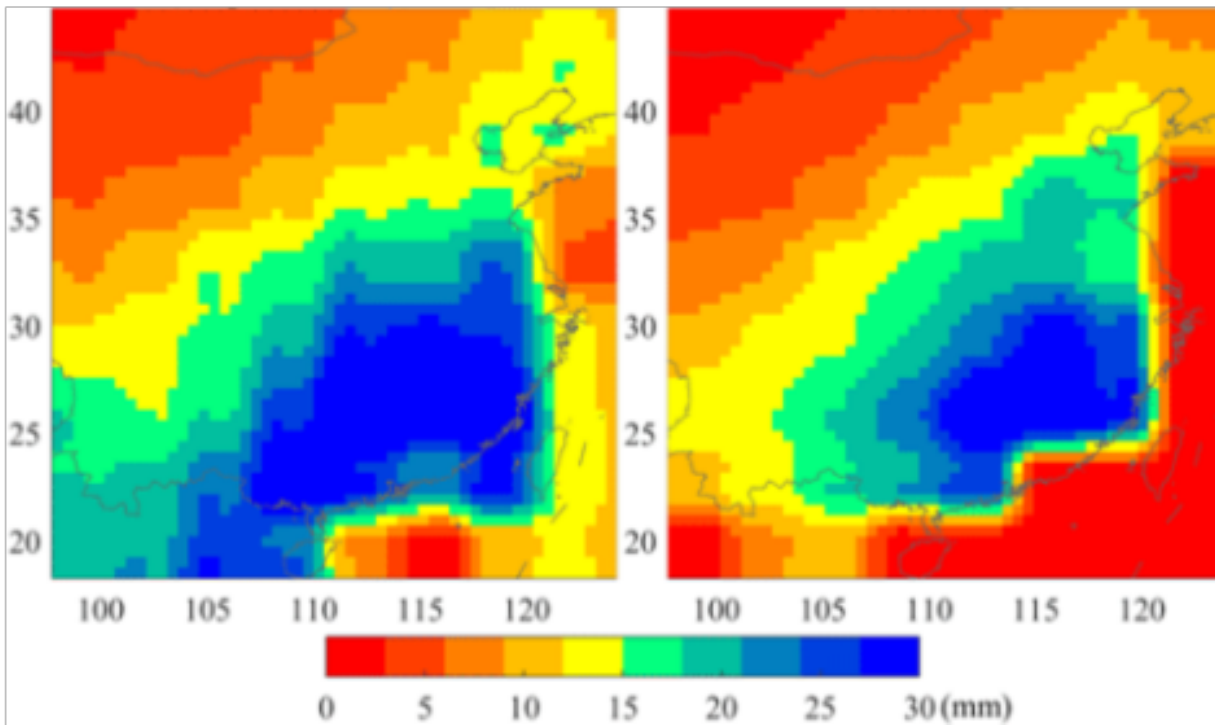
Uncertainty Estimation

Uncertainty Applications

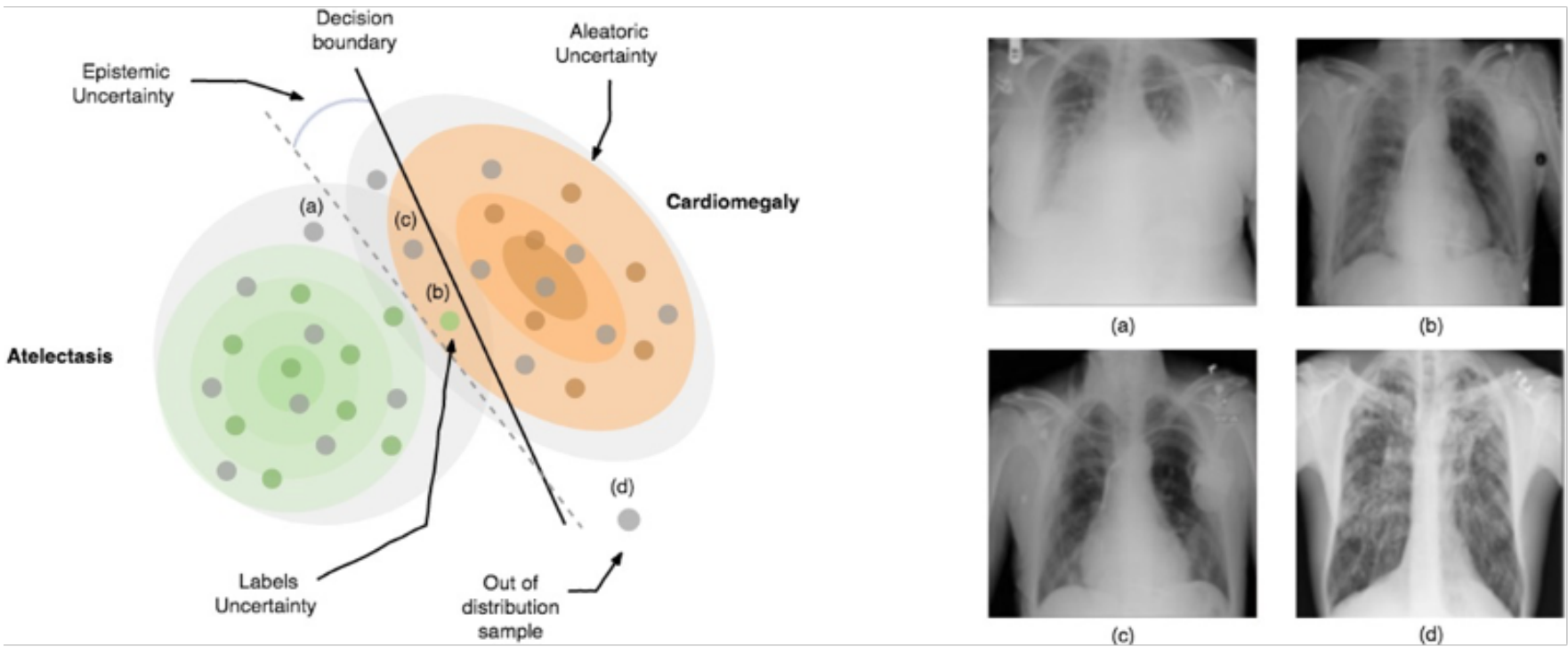
Autonomous Driving



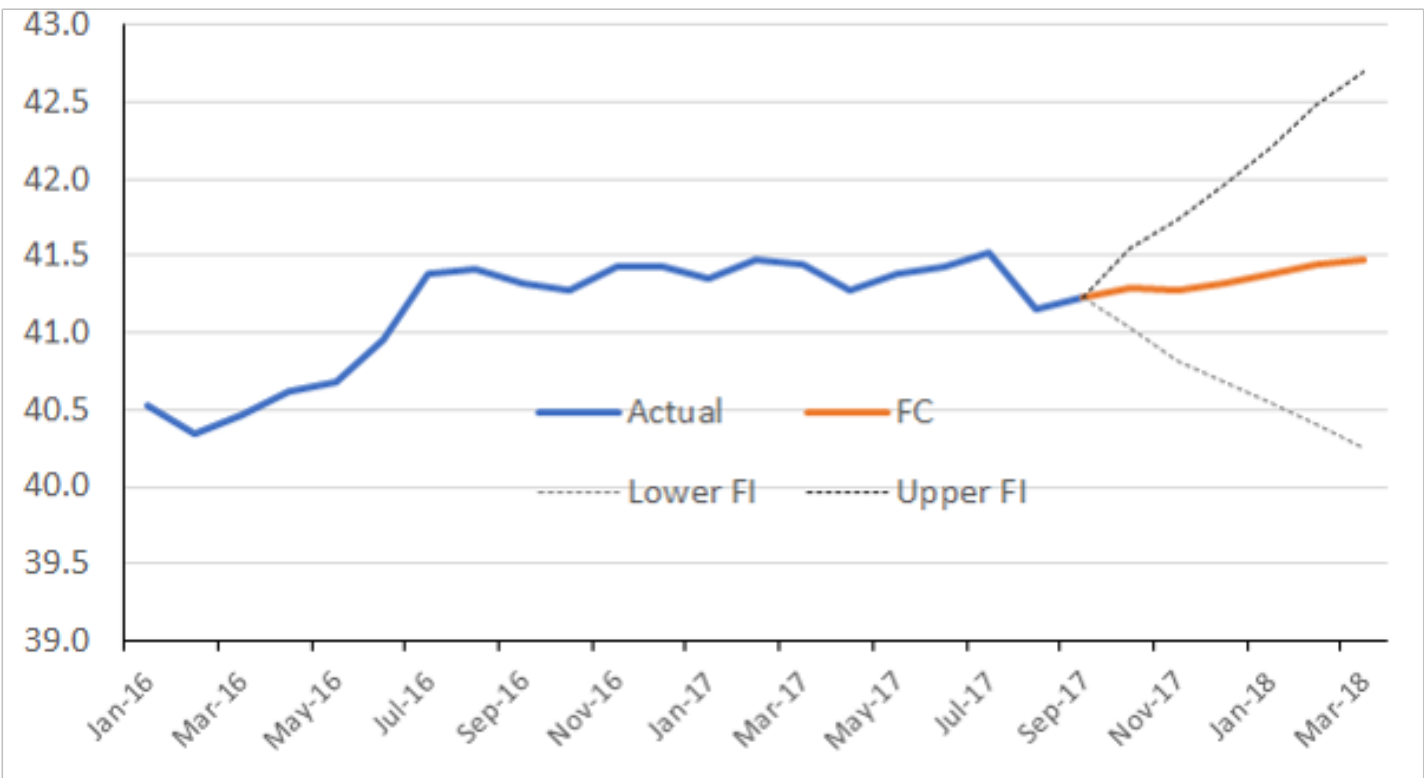
Weather Forecasting



Medical Imaging



Finance Applications



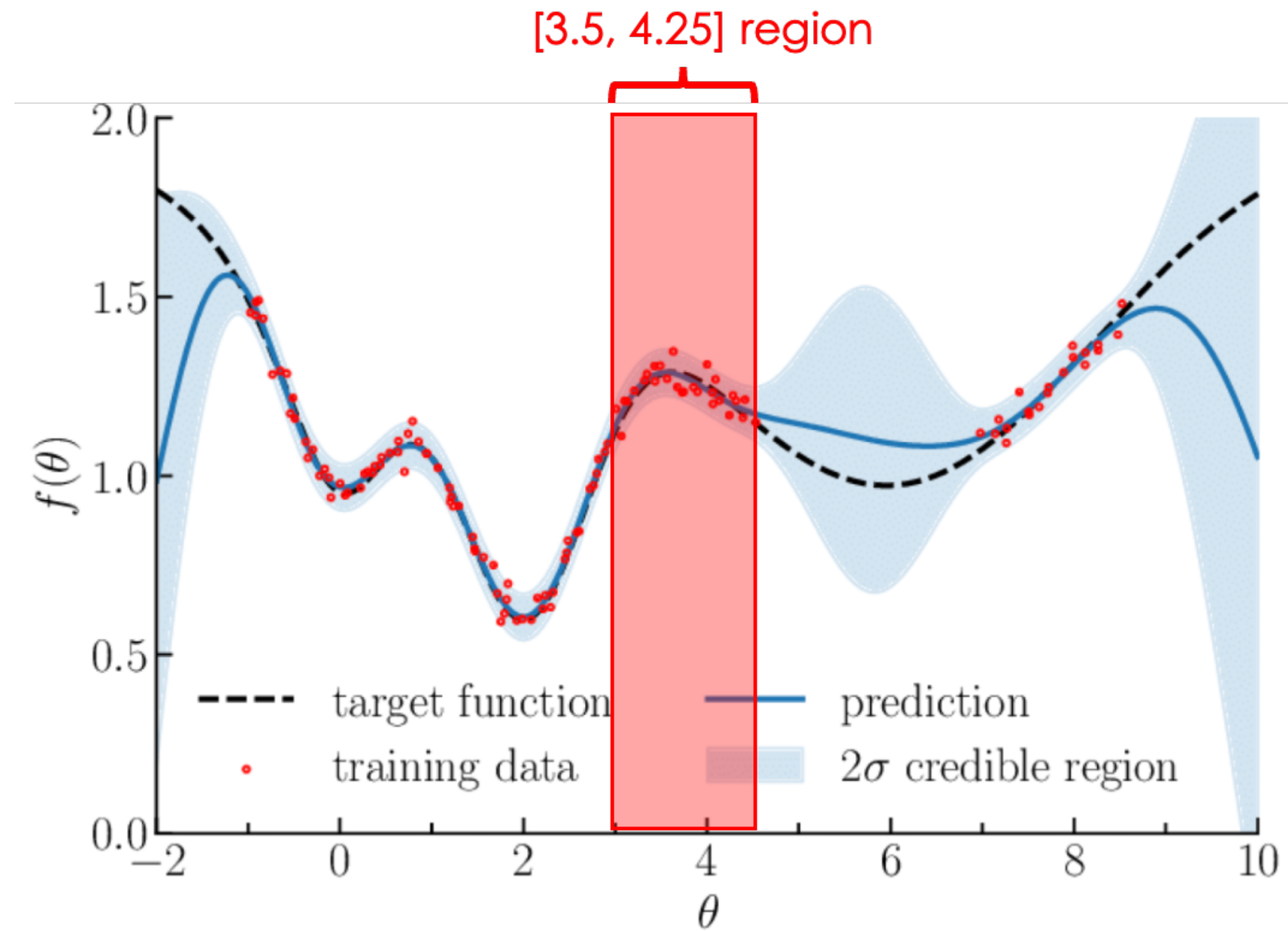
Uncertainty Estimation

Uncertainty Types

- **Aleatoric Uncertainty:** This type of uncertainty is related to inherent randomness in the data. It represents the noise inherent in observations, which cannot be reduced even with more data.

Uncertainty Estimation

Uncertainty Types



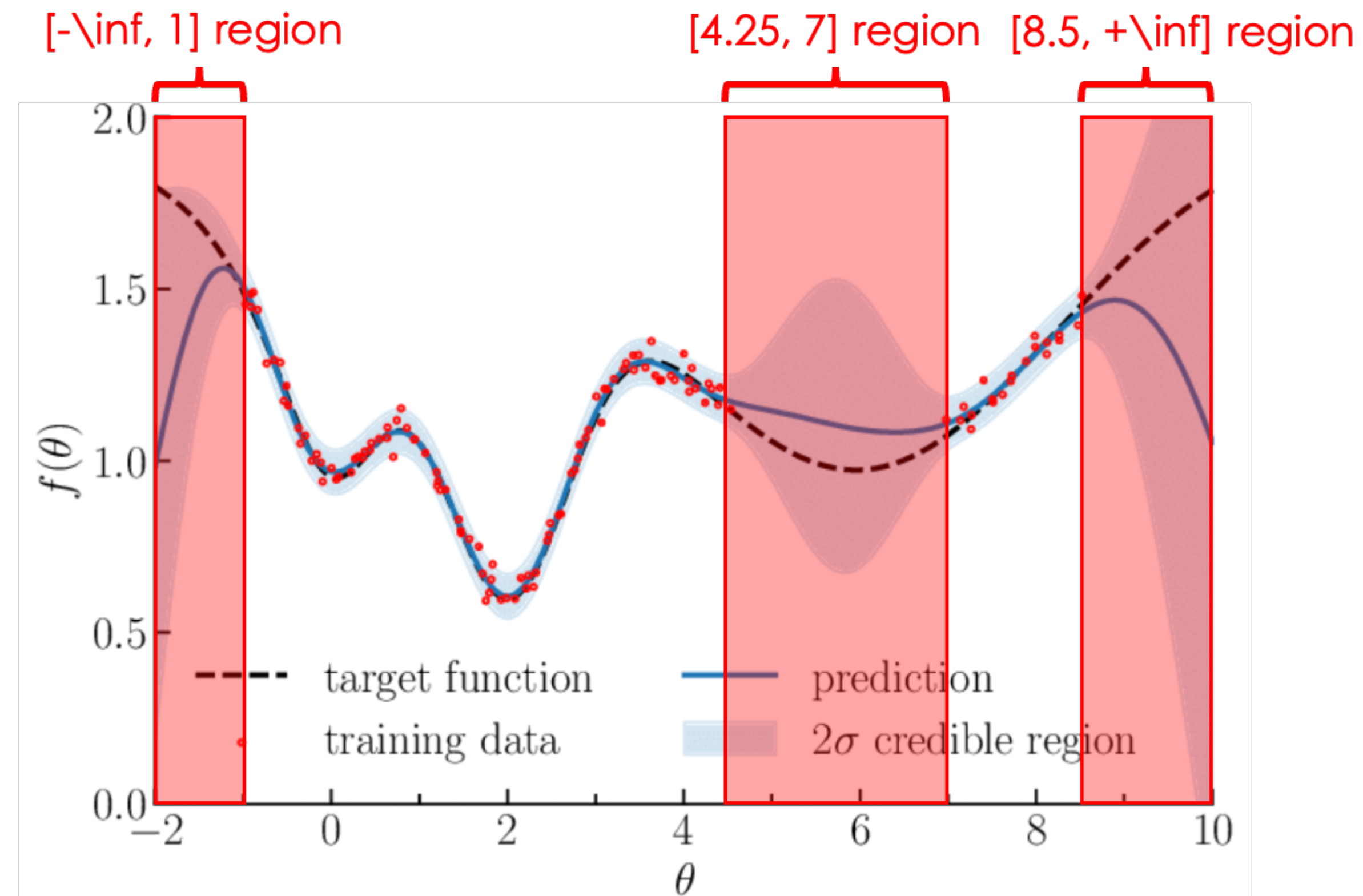
Uncertainty Estimation

Uncertainty Types

- **Aleatoric Uncertainty:** This type of uncertainty is related to inherent randomness in the data. It represents the noise inherent in observations, which cannot be reduced even with more data.
- **Epistemic Uncertainty:** Caused by limited data or model design, it reflects uncertainty in model parameters and can be reduced with more data or better models.

Uncertainty Estimation

Uncertainty Types

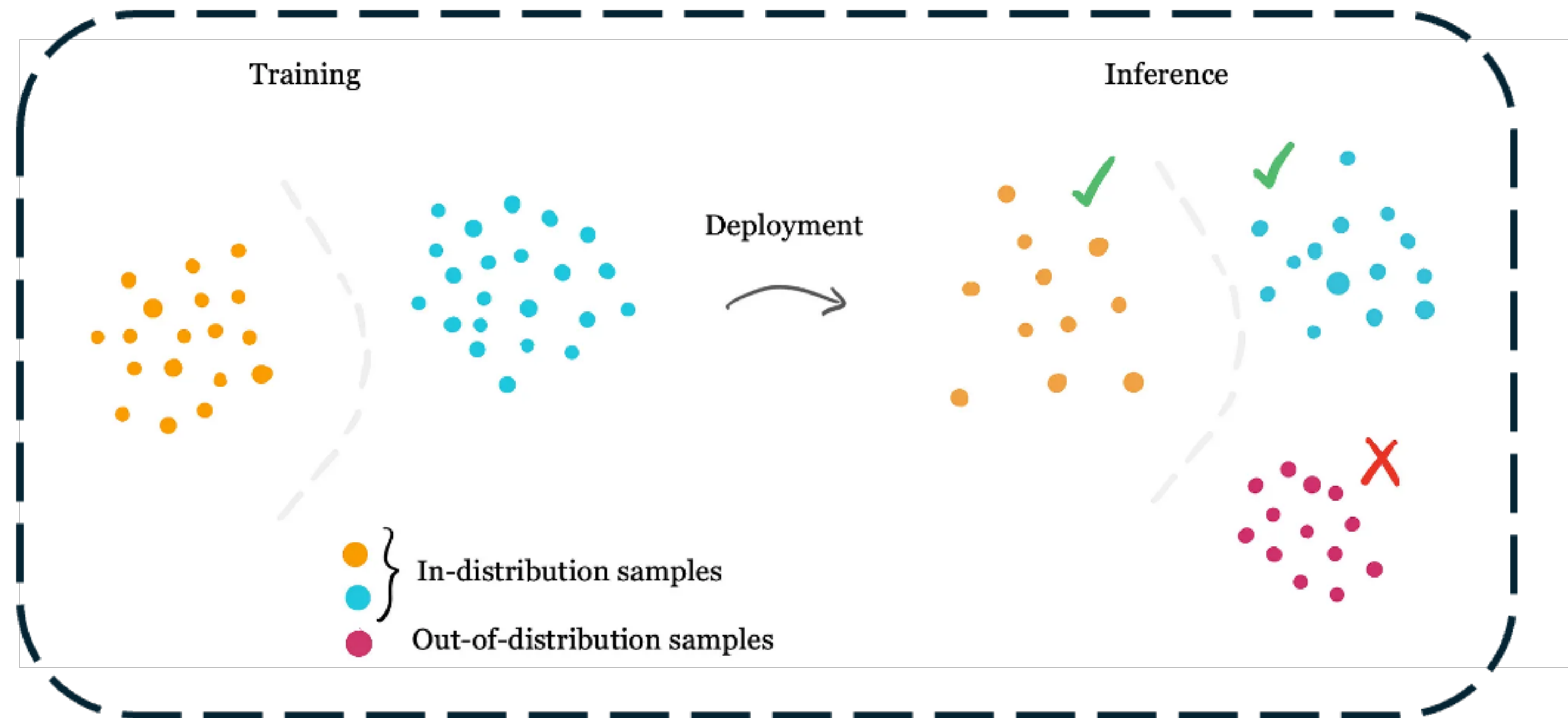


Uncertainty Estimation

Uncertainty Applications

Safety / Robustness:

- Robust predictions
- OOD detection
- Human verification



Uncertainty Estimation

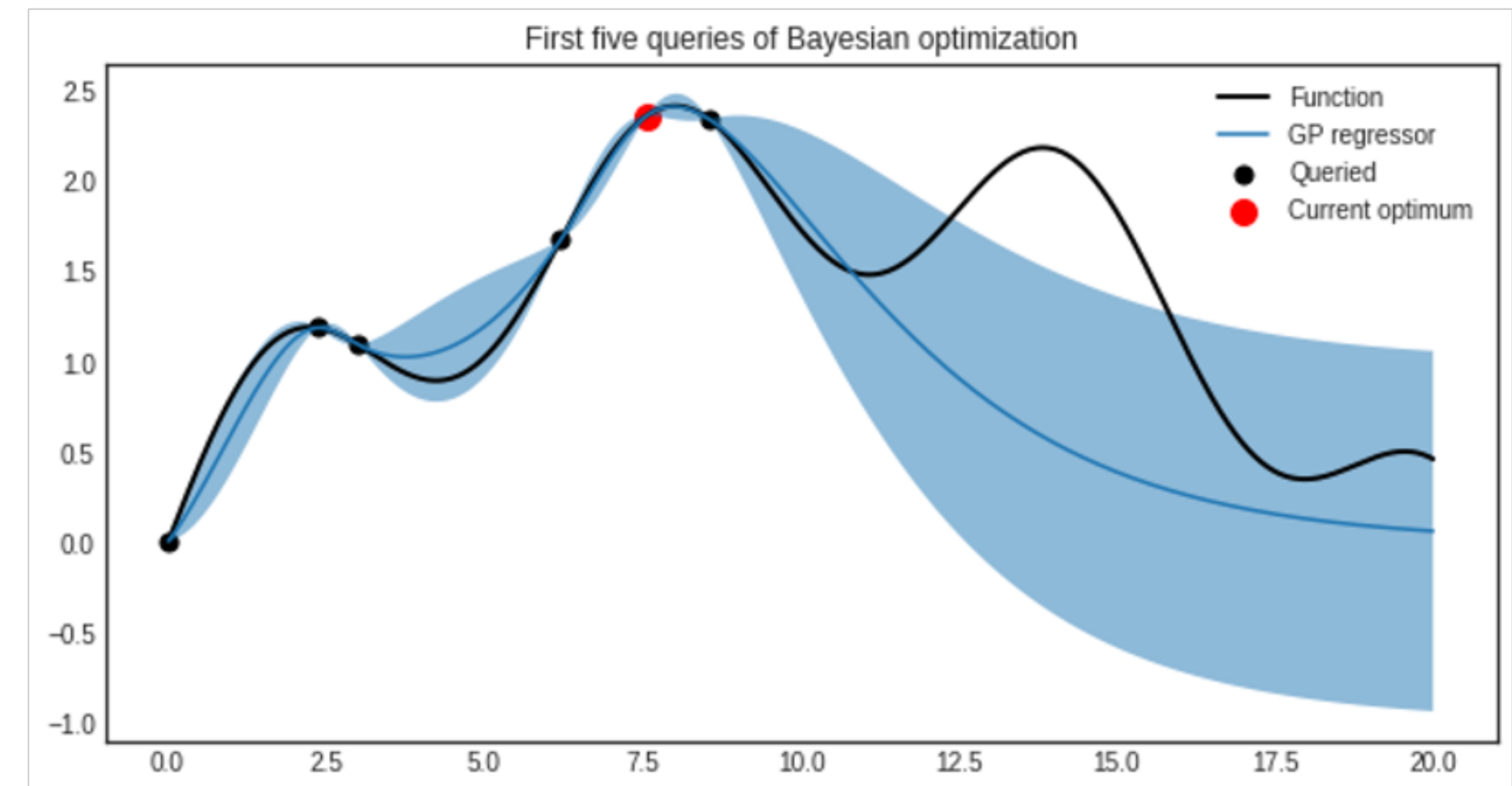
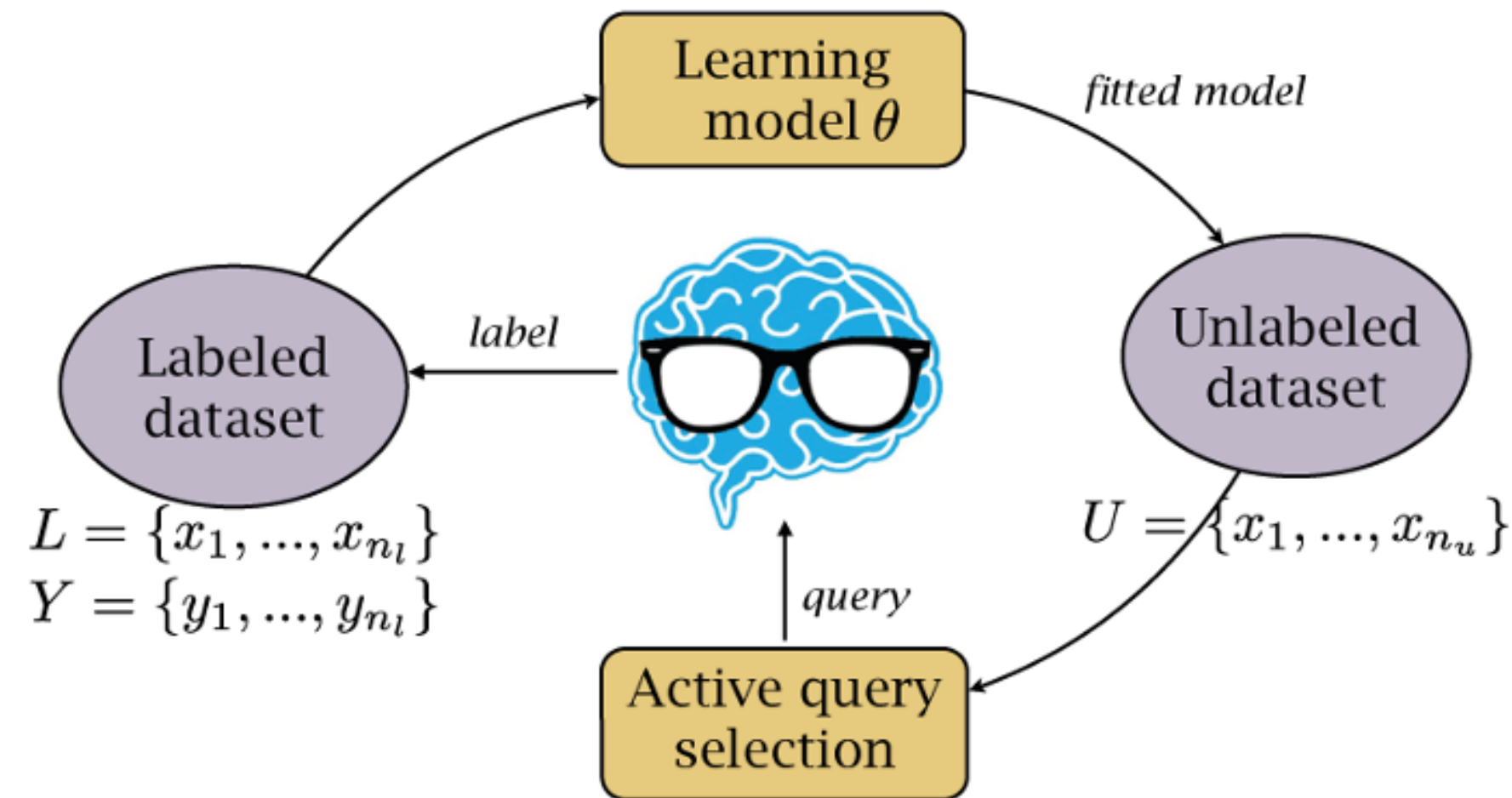
Uncertainty Applications

Safety / Robustness:

- Robust predictions
- OOD detection
- Human verification

Exploration:

- Active learning
- Bayesian Optimization
- RL Exploration



Uncertainty Estimation

Uncertainty Methods

- Uncertainty Quantification (UQ) is highly important in classical machine learning, and many traditional ML methods account for it.

Uncertainty Estimation

Uncertainty Methods

- Uncertainty Quantification (UQ) is highly important in classical machine learning, and many traditional ML methods account for it.
- However, most existing approaches are not well-suited for deep learning, highlighting the need for the development of new methods

Uncertainty Estimation

Modern Methods (Ensembling)

Popular approaches:

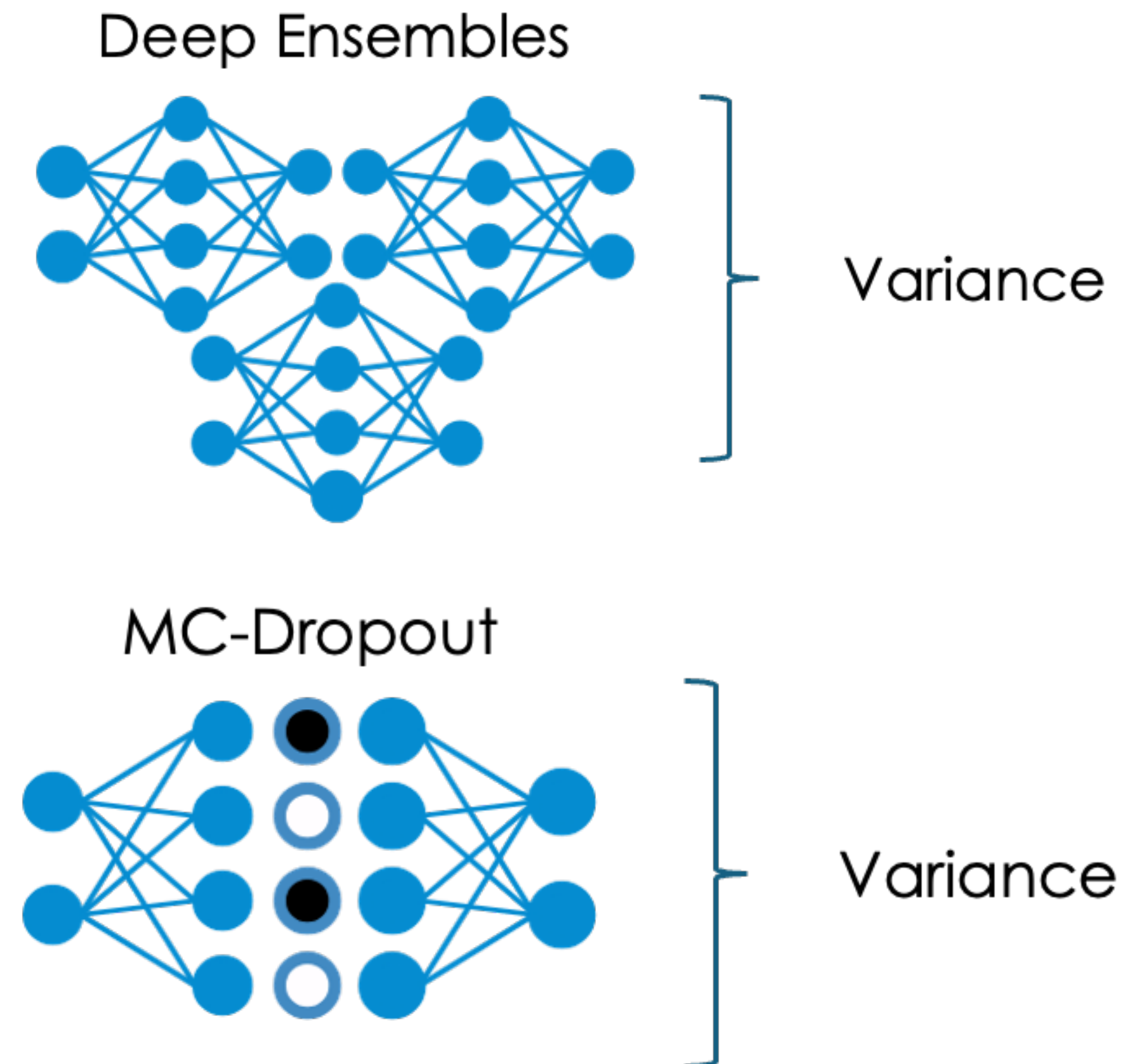
- Ensembles
- MC-Dropout

Drawbacks:

- Require multiple inferences (N times)

Advantages:

Best Uncertainty quality (Ensembles)



Uncertainty Estimation

Modern Methods (Ensembling)

- Deep Ensembles (Lakshminarayanan et al.)
- Snapshot Ensembles (Huang et al.)
- MC-Dropout (Gal et al.)
- BatchEnsemble (Wen et al.)
- Packed-Ensembles (Laurent et al.)
- MIMO (Rame et al.)
- Fast Geometric Ensembling (Garipov et al)
- ABNN (Laurent et al.)
- Depth Uncertainty (Antorán et al.)
- ...
-

Uncertainty Estimation

Modern Methods (Ensembling)

Ensembling methods are **difficult to train**, **slow**, and **demanding** in terms of **memory** and **computation**, making them impractical for many real-world applications, such as robotics or finance.

Can we do better?

Uncertainty Estimation

Modern Methods (Sampling-Free)

Popular approaches:

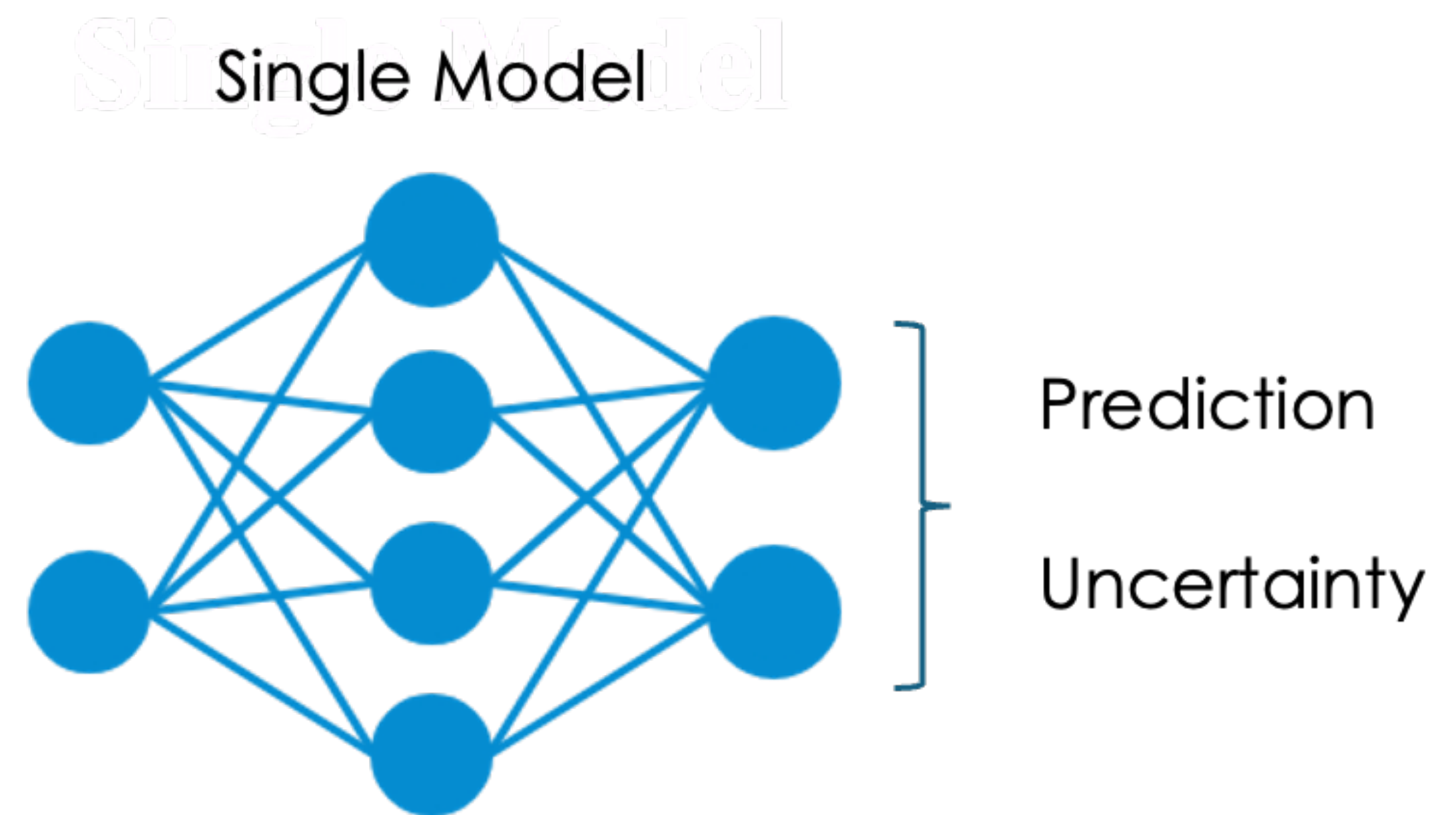
- SNGP
- Predicting uncertainty as output

Drawbacks:

- Worse uncertainty quality
- Often limited to a particular task

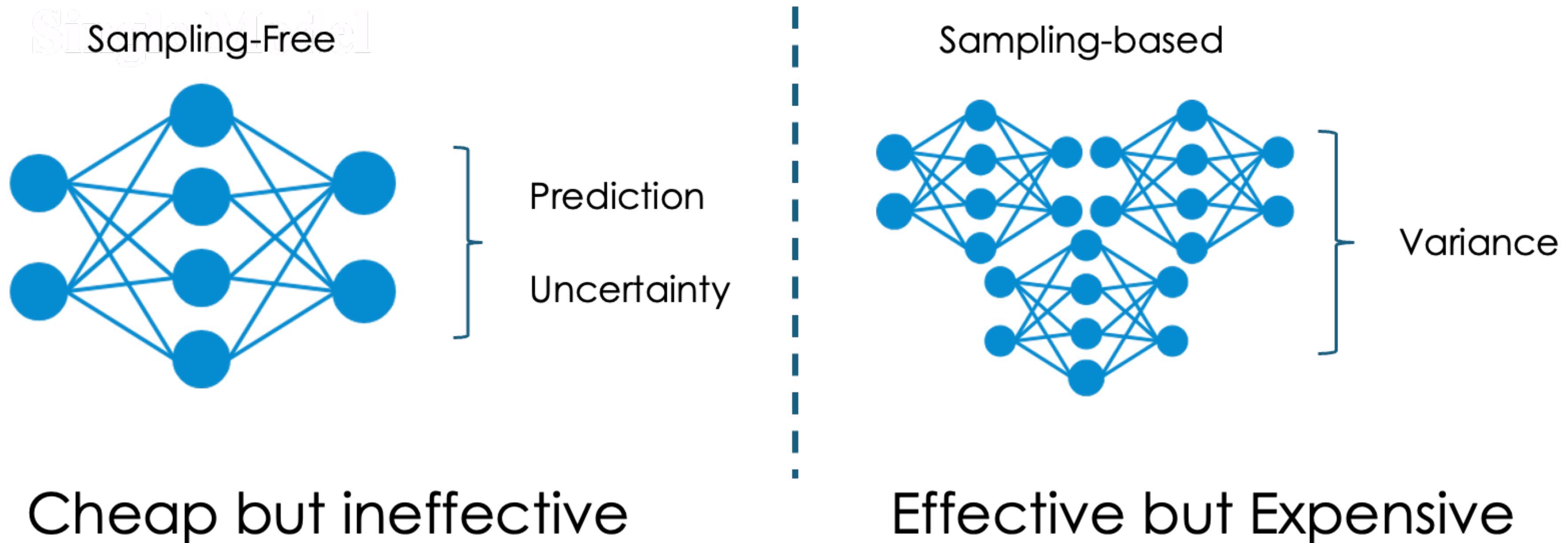
Advantages:

- Fast inference, cheap to train



Uncertainty Estimation

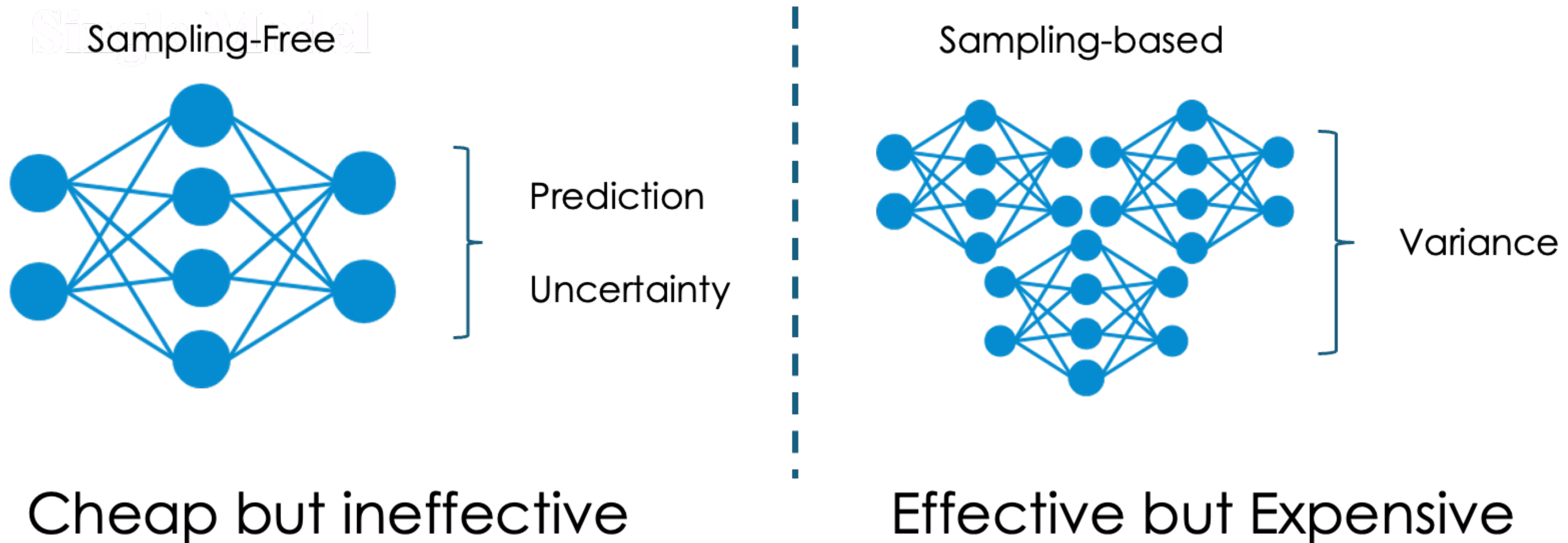
Modern Methods (Trade off)



Can we do both?

Uncertainty Estimation

Modern Methods (Trade off)



This is precisely the focus of cutting-edge research efforts in the fields of safety and uncertainty estimation.

Thank you for your attention!
Questions?